

TextReIDNet

Diseño e implementación de un modelo
de reidentificación de personas
basado en texto

Documento técnico — Proyecto `textreid-train`

14 de septiembre de 2026

Resumen

Este libro documenta en profundidad el proyecto `textreid-train`, una implementación limpia y enfocada al entrenamiento del modelo **TextReIDNet** para *reidentificación de personas basada en texto*. El modelo combina un backbone visual EfficientNet-B0 con una red de lenguaje basada en *BERT* + GRU bidireccional, fusionando ambas modalidades en un espacio de *embedding* conjunto de dimensión 1024. El entrenamiento se realiza sobre el dataset **CUHK-PEDES**.

El documento parte del paper original de Agyeman *et al.* (*IEEE Access*, 2024) – “Reidentificación descentralizada de personas basada en texto en redes multicámara” – cuya versión traducida al español se encuentra en `docs/paper.md`, y desciende hasta la última línea de cada archivo del repositorio: explica qué hace cada clase, qué forma tienen los tensores en cada capa, dónde se define cada parámetro de configuración y por qué se eligieron esos valores.

Está pensado como material de referencia para quien quiera entender, modificar, depurar o reentrenar el modelo. No se asume experiencia previa con *re-ID*, pero sí familiaridad básica con *PyTorch* y *deep learning*.

Índice general

1. Prólogo	1
1.1. ¿A quién está dirigido?	1
1.2. Qué asume el lector	1
1.3. Convenciones tipográficas	1
1.4. Estructura del libro	2
1.5. Código fuente	2
2. Introducción a la reidentificación de personas	3
2.1. ¿Qué es la reidentificación de personas?	3
2.2. Reidentificación basada en texto	3
2.3. Desafíos de los sistemas existentes	3
2.4. La solución propuesta: U-TextReIDNet	4
2.5. El dataset CUHK-PEDES	4
2.6. Lo que este libro no cubre	4
3. U-TextReIDNet: arquitectura teórica	5
3.1. Formulación del problema	5
3.2. Componentes principales	5
3.2.1. Red visual	5
3.2.2. Red de lenguaje	6
3.2.3. Componente de coincidencia de embedding conjunto	6
3.3. Diagrama de bloques	7
3.4. Dimensiones en cada etapa: tabla resumen	8
3.5. Por qué estas elecciones	8
3.6. Lo que cambia en este proyecto	9
4. Sistema descentralizado multicámara	10
4.1. Arquitectura del sistema	10
4.2. Flujo operativo	10
4.3. Implementación de la estación de control	11
4.4. Por qué este proyecto no incluye el sistema	11
4.5. Rendimiento reportado en el paper	12
4.6. Lo que este proyecto sí hace	12
5. El dataset CUHK-PEDES	13
5.1. El dataset en cifras	13
5.2. Formato en disco	13
5.2.1. El archivo <code>reid_raw.json</code>	14
5.3. Cómo se cargan los datos	14
5.3.1. El loader HF: <code>CUHKPEDESHF</code>	14
5.3.2. El loader original: <code>CUHKPEDES</code>	15

5.4.	El dataloader unificado	15
5.5.	La clase <code>ImageTextDataset</code>	15
5.6.	Preprocesamiento de imagen: <code>place_image_on_canvas</code>	16
5.7.	Preprocesamiento de texto	17
5.8.	Normalización de imagen	17
5.9.	El cuaderno de validación	18
5.10.	Dataset augmentation	18
5.11.	Resumen del pipeline de datos	18
6.	Configuración del proyecto	19
6.1.	La clase <code>DotDict</code>	19
6.2.	Función principal: <code>sys_configuration</code>	19
6.3.	Tabla maestra de parámetros	19
6.4.	Override por línea de comandos	22
6.5.	Configuración por plataforma	22
6.6.	Reproducibilidad	22
6.7.	Ejemplo de inspección	23
7.	Red visual: EfficientNet-B0 y convolución separable	24
7.1.	¿Qué es EfficientNet?	24
7.1.1.	El problema	24
7.1.2.	Compound scaling	24
7.1.3.	¿Qué es MBConv?	24
7.1.4.	Layout de B0	25
7.2.	¿Qué es la convolución separable en profundidad (DSC)?	25
7.2.1.	El problema	25
7.2.2.	La factorización	25
7.2.3.	Visualización	26
7.3.	¿Qué es <code>AdaptiveMaxPool2d</code> ?	26
7.4.	Código: la clase <code>VisualNetwork</code>	26
7.5.	Código: <code>DepthwiseSeparableConv</code>	27
7.6.	Etapas con shapes exactas (input 512×512)	28
7.7.	Flujo de tensores	28
7.8.	Parámetros	29
8.	Red de lenguaje: BERT, embeddings, BiGRU y pack-padded	30
8.1.	¿Qué es BERT y por qué sólo su tokenizer?	30
8.1.1.	BERT	30
8.1.2.	¿Por qué no usar BERT entero?	30
8.1.3.	Cómo funciona el tokenizer de BERT	30
8.2.	¿Qué es un embedding?	31
8.3.	¿Qué es un padding?	31
8.4.	¿Qué es una GRU?	31
8.5.	¿Qué es pack-padded-sequence?	31
8.5.1.	El problema	31
8.5.2.	La solución	32
8.6.	Código: <code>GRULanguageNetwork</code>	32
8.7.	Código: <code>do_RNN_computation</code>	33
8.8.	Código: <code>BERTTokenizer</code>	33
8.9.	Código: <code>pad_tokens</code>	34
8.10.	Flujo de tensores	34
8.11.	Parámetros	35

9. TextReIDNet: joint embedding visual-textual	36
9.1. ¿Qué es un joint embedding?	36
9.1.1. El concepto	36
9.2. ¿Qué es la similitud coseno?	36
9.3. ¿Por qué un DSC compartido?	36
9.4. ¿Qué hace <code>squeeze</code> y <code>contiguous</code> ?	37
9.4.1. <code>squeeze(dim)</code>	37
9.4.2. <code>contiguous()</code>	37
9.5. ¿Por qué dimensión 1024?	37
9.6. Código: Constructor <code>__init__</code>	37
9.7. Código: <code>forward</code>	38
9.8. Código: <code>image_embedding</code>	38
9.9. Código: <code>text_embedding</code>	39
9.10. Diagrama del flujo completo	40
9.11. Inicialización de pesos	40
9.12. Inferencia: extraer embeddings	40
10. Funciones de pérdida: triplet y cross-entropy	42
10.1. ¿Qué es un embedding y por qué necesita pérdidas?	42
10.2. ¿Qué es la triplet loss?	42
10.2.1. Concepto	42
10.2.2. ¿Por qué el margen?	42
10.3. ¿Qué es semi-hard mining?	43
10.3.1. Los tres tipos de negativos	43
10.3.2. Criterio usado en este proyecto	43
10.4. ¿Qué es la cross-entropy?	43
10.4.1. Clasificación como identidad	43
10.4.2. PyTorch lo hace solo	43
10.5. ¿Por qué simétrica ($i2t + t2i$)?	43
10.6. Pérdida total	43
10.7. Código: <code>calculate_similarity</code>	44
10.8. Código: <code>RankingLoss</code>	44
10.9. Código: <code>IdentityLoss</code>	45
10.10. Uso conjunto en <code>train.py</code>	46
11. Entrenamiento: optimizador, scheduler y mixed precision	47
11.1. ¿Qué es AdamW?	47
11.1.1. Adam básico	47
11.1.2. AdamW	47
11.2. ¿Qué es un LR scheduler?	48
11.3. ¿Qué es mixed precision (FP16/BF16)?	48
11.3.1. Precisiones disponibles	48
11.3.2. El problema del FP16	48
11.3.3. Solución: GradScaler	48
11.3.4. Autocast	48
11.3.5. GPU vs CPU	49
11.4. ¿Qué hace <code>optimizer.zero_grad()</code> ?	49
11.5. ¿Cómo se guarda un checkpoint?	49
11.6. Código: <code>train.py</code> main + loop	49
11.7. Estimación de tiempo	52
11.8. Caveats	52

12. Evaluación: extracción de features y métricas	53
12.1. ¿Qué es la inferencia (evaluación)?	53
12.2. ¿Por qué <code>@torch.no_grad()</code> ?	53
12.3. ¿Qué es <code>model.eval()</code> ?	53
12.4. Galería vs query: convención de este proyecto	53
12.5. Código: extracción de features de imágenes	54
12.6. Código: extracción de features de textos	54
12.7. Código: similitud coseno	55
12.8. Código: pipeline completo de evaluación	55
12.9. Uso desde la terminal	56
12.10. Resultados típicos	57
12.11. Limitaciones	57
13. Métricas: Top-k, AP, mAP y CMC	58
13.1. El problema de evaluación	58
13.2. Top-k accuracy (CMC)	58
13.3. Average Precision (AP)	59
13.4. mAP (mean Average Precision)	61
13.5. CMC: el vector completo	61
13.6. Interpretación de números	63
13.7. Ejemplo numérico completo	63
13.8. Resumen	64
14. Apéndices	65
14.1. Glosario	65
14.2. Tabla maestra de parámetros del modelo	65
14.3. Tabla maestra de hiperparámetros de entrenamiento	66
14.4. Resultados reportados en el paper	67
14.5. Mapa de archivos	67
14.6. Cheatsheet de Bash	68
14.7. Referencias	68

Índice de figuras

3.1. Arquitectura de U-TextReIDNet. Las cajas con relleno gris son las que este proyecto implementa; las cajas con relleno claro son las del paper que no están en el código (R-MHParsNet y sistema distribuido).	7
4.1. Topología del sistema descentralizado. Cada cámara corre U-TextReIDNet localmente; sólo transmite vídeo cuando su puntuación supera un umbral y es la mejor de todas.	10
5.1. Pipeline de datos desde el JSON hasta el batch que consume <code>train.py</code>	18
7.1. Comparación entre convolución estándar y DSC.	26
9.1. Flujo completo de tensores. Notar que el último DSC es compartido entre ambas ramas.	40

Índice de tablas

2.1. Estadísticas del dataset CUHK-PEDES original.	4
3.1. Dimensiones de los tensores en cada etapa del modelo. B denota el tamaño de batch. Las columnas “Etapa paper” y “Etapa proyecto” difieren únicamente en la resolución espacial de entrada.	8
4.1. Tiempos de inferencia reportados en el paper (media sobre 20 iteraciones).	12
4.2. Tiempos de inferencia para 10 imágenes de galería.	12
5.1. Diferencias entre la versión original y la versión HuggingFace de CUHK-PEDES.	13
6.1. Parámetros del modelo de lenguaje.	20
6.2. Parámetros del modelo visual.	20
6.3. Parámetros de entrenamiento.	21
6.4. Parámetros de funciones de pérdida.	21
6.5. Paths del proyecto.	22
7.1. Las 9 etapas de EfficientNet-B0.	25
7.2. Etapas con shapes para input 512×512 en este proyecto.	28
7.3. Parámetros por componente visual.	29
8.1. Parámetros de la red de lenguaje.	35
12.1. Resultados esperados según el README.	57
13.1. Ejemplo de ranking para una query con 3 positivos.	59
13.2. Resultados esperados según el README y el paper.	63
13.3. Resumen de las cuatro métricas.	64
14.1. Recuento completo de parámetros. N_c = número de identidades (clases) en el dataset de entrenamiento. Para CUHK-PEDES, $N_c = 11\,003$	66
14.2. Hiperparámetros de entrenamiento.	66
14.3. Resultados del paper original.	67

List of Listings

Capítulo 1

Prólogo

Este libro es una guía exhaustiva del proyecto **textreid-train**: un repositorio limpio, enfocado exclusivamente al entrenamiento y la evaluación del modelo **TextReIDNet** para *reidentificación de personas basada en texto*. No es un manual de uso ni un *quickstart*; es un documento de referencia técnica que recorre cada archivo del proyecto y explica, capa por capa, qué hace y por qué.

1.1 ¿A quién está dirigido?

- **Estudiantes o practitioners de visión por computador** que quieran entender un sistema real de *re-ID* basado en texto, desde la matemática del paper hasta los detalles de implementación.
- **Investigadores** que busquen reproducir, comparar o extender el modelo.
- **Ingenieros** que tengan que mantener, depurar o desplegar este código en producción.

1.2 Qué asume el lector

- Familiaridad básica con **PyTorch** (tensores, `nn.Module`, `DataLoader`).
- Conocimiento general de *deep learning*: redes convolucionales, recurrentes, funciones de pérdida y optimización.
- Conocimiento elemental de procesamiento de lenguaje natural: *tokenización*, *embeddings*, GRU.

No se asume experiencia previa con *reidentificación de personas*, *EfficientNet*, *BERT* ni métricas como MAP.

1.3 Convenciones tipográficas

- Los nombres de archivos se escriben en `\texttt{monoespaciado}` y se citan con su ruta relativa al raíz del proyecto: `model/textreidnet.py`.
- Las referencias a líneas específicas se hacen con el formato `archivo.py:L`. Por ejemplo, `config.py:42` apunta a la línea 42 de `config.py`.
- Las formas de los tensores se muestran entre paréntesis en color azul: `((B, 1280, 16, 16))` significa *batch* $\times$ *canales* $\times$ *alto* $\times$ *ancho*.
- Los parámetros de configuración aparecen en **negrita mono**.
- Los listados de código van numerados y muestran sólo la región relevante. La numeración coincide con la del archivo original.

1.4 Estructura del libro

El libro se divide en cuatro partes:

1. **Fundamentos teóricos** (Caps. 2 y 3): contexto del problema, formulación matemática y arquitectura de U-TextReIDNet según el paper.
2. **Sistema descentralizado** (Cap. 4): cómo se despliega el modelo en una red de cámaras reales.
3. **Implementación** (Caps. 5–12): dataset, configuración, redes, pérdidas, entrenamiento y evaluación. Es el grueso del libro.
4. **Apéndices** (Cap. 14): glosario, tablas de referencia rápida.

1.5 Código fuente

Todos los listados del libro provienen de los archivos reales del repositorio `textreid-train`. Se recomienda abrir el repositorio en paralelo a la lectura para poder navegar entre el PDF y el código sin perderse.

textreid-train/
model/ · datasets/ · evaluation/
utils/ · scripts/ · config.py

Sobre este documento

Versión: 1.0

Fecha de compilación: 14 de septiembre de 2026

Paper de referencia: Agyeman *et al.*, “Decentralized Text-Based Person Re-Identification in Multicamera Networks”, *IEEE Access*, noviembre de 2024. DOI: [10.1109/ACCESS.2024.3501382](https://doi.org/10.1109/ACCESS.2024.3501382).

Traducción del paper al español: docs/paper.md (552 líneas).

Conviene leer este libro con un navegador de código abierto. Cada vez que veas una referencia como `model/textreidnet.py:13`, podrás saltar directamente al constructor de la clase `TextReIDNet`.

Capítulo 2

Introducción a la reidentificación de personas

Este capítulo sienta las bases del problema que el modelo intenta resolver. Toda la información proviene del paper traducido al español en `docs/paper.md`.

2.1 ¿Qué es la reidentificación de personas?

La *reidentificación de personas* (en inglés *person re-identification*, abreviada re-ID) es la tarea de reconocer a la *misma* persona en distintas fotografías capturadas por cámaras diferentes – o por la misma cámara en distintos momentos. La consulta de búsqueda puede ser:

- una **imagen** (*image-to-image re-ID*),
- una **secuencia de vídeo**,
- una **descripción textual** (*text-based re-ID*).

A diferencia del reconocimiento facial, la re-ID no depende de características faciales – que pueden ser de baja calidad o estar restringidas por privacidad – sino de atributos visibles como ropa, color, tipo de cuerpo o accesorios [?]. Esto la hace especialmente útil en entornos concurridos donde es difícil obtener caras nítidas.

2.2 Reidentificación basada en texto

La re-ID basada en texto (*text-based person re-identification*) consiste en recuperar la imagen de una persona a partir de una descripción en lenguaje natural. Por ejemplo, ante la consulta:

“Una mujer lleva una camiseta negra con letras rosas. Viste pantalones cortos color caqui y zapatos blancos.”

el sistema debe devolver, de una galería de miles de imágenes, las que correspondan a esa persona.

Esta modalidad tiene dos ventajas prácticas frente a la re-ID basada en imagen:

1. Las consultas son más **accesibles**: cualquiera puede describir a una persona, pero no siempre se tiene una foto reciente.
2. Resuelve **problemas de privacidad**: en muchas jurisdicciones capturar y almacenar caras está restringido, mientras que las descripciones textuales son menos invasivas.

2.3 Desafíos de los sistemas existentes

Los algoritmos *state-of-the-art* de re-ID por texto suelen ser modelos enormes (cientos de millones de parámetros) que priorizan precisión sobre eficiencia. Esto trae tres problemas en aplicaciones reales:

- **Requisitos computacionales altos:** no se pueden ejecutar en dispositivos embebidos como una cámara inteligente.
- **Dependencia de arquitecturas centralizadas:** todas las cámaras envían vídeo a un servidor central que ejecuta la identificación. Esto genera cuellos de botella, problemas de escalabilidad y consumo elevado de ancho de banda.
- **Imágenes pre-recortadas:** la mayoría de los algoritmos asumen que la imagen de entrada contiene una sola persona ya recortada, lo cual es irreal en escenas concurridas.

2.4 La solución propuesta: U-TextReIDNet

El paper de Agyeman *et al.* [?] propone **U-TextReIDNet** (Unified Text Re-Identification Network), un modelo:

- **Preciso:** alcanza **54,02 %** Top-1 en CUHK-PEDES y **38,45 %** Top-1 en RSTPReid.
- **Ligero:** solo **38,77 millones** de parámetros totales.
- **Eficiente:** corre en tiempo real en un *NVIDIA Jetson Nano*.
- **Unificado:** integra la detección de personas con la identificación, eliminando la necesidad de pre-recortar.

A diferencia de sus predecesores *MHParsNet* (detección) y *TextReIDNet* (identificación), que eran módulos separados, U-TextReIDNet los une en un solo modelo end-to-end.

2.5 El dataset CUHK-PEDES

Este proyecto entrena sobre **CUHK-PEDES** [?], uno de los benchmarks estándar para re-ID por texto. Estadísticas según el paper:

Tabla 2.1: Estadísticas del dataset CUHK-PEDES original.

Split	Imágenes	Descripciones	Personas
Train	34 054	68 126	11 003
Val	3 078	6 158	1 000
Test	3 074	6 156	1 000
Total	40 206	80 440	13 003

Cada imagen tiene asociadas *dos* descripciones textuales escritas por anotadores humanos. En este proyecto se usa una versión alternativa descargada de *HuggingFace* (PeterPanTheGenius/CUHK-PEDES) que tiene una sola descripción por imagen pero está disponible sin necesidad de solicitar acceso por email. Los detalles están en el Capítulo 5.

2.6 Lo que este libro no cubre

U-TextReIDNet completo (paper) incluye dos componentes:

- **R-MHParsNet:** detector de múltiples personas (*FPN* + EfficientNet-B0 + cabezas de segmentación). Tiene **6,49M** parámetros y NO está implementado en este proyecto.
- **TextReIDNet:** el modelo de identificación imagen-texto (EfficientNet-B0 + BERT + Bi-GRU). Éste es el núcleo del proyecto `textreid-train`.

Este libro se centra exclusivamente en **TextReIDNet** y su entrenamiento. Para la parte descentralizada (Jetson Nano, ZMQ, MQTT, Flask) se da contexto en el Capítulo 4 pero sin entrar en código, ya que el repositorio no la incluye.

Capítulo 3

U-TextReIDNet: arquitectura teórica

Este capítulo desglosa la arquitectura completa del modelo U-TextReIDNet tal como se describe en `docs/paper.md` (§III-A, líneas 146–178). Aunque el repositorio `textreid-train` implementa solo la parte de identificación (*TextReIDNet*), entender la versión unificada ayuda a comprender las elecciones de diseño.

3.1 Formulación del problema

Dado un fotograma de vídeo con n personas distintas $\mathbf{V} \in \mathbb{R}^{H \times W \times 3}$ y una descripción textual $\mathbf{T}$ en lenguaje natural, los objetivos son:

1. **Detectar y segmentar** las n instancias humanas en $\mathbf{V}$, produciendo una galería $\mathbf{I} = \{\mathbf{I}_1, \dots, \mathbf{I}_n\}$. Si $n = 0$, la galería está vacía.
2. **Identificar** la imagen $\mathbf{I}^* \in \mathbf{I}$ que mejor se ajusta a $\mathbf{T}$ mediante una función de coincidencia multimodal $\mathcal{M}$.

La función $\mathcal{M}$ usa las *incrustaciones* (embeddings) visuales y textuales:

$$\mathbf{I}' = F_i(\mathbf{I}) \quad \text{embedding visual} \quad (3.1)$$

$$\mathbf{T}' = F_t(\mathbf{T}) \quad \text{embedding textual} \quad (3.2)$$

y calcula su similitud con la métrica del coseno:

$$\text{sim}(\mathbf{I}', \mathbf{T}') = \frac{\mathbf{I}' \cdot \mathbf{T}'}{\|\mathbf{I}'\| \|\mathbf{T}'\|} \quad (3.3)$$

El resultado óptimo es la imagen que maximiza esa similitud.

3.2 Componentes principales

U-TextReIDNet se compone de tres bloques:

1. **Red visual:** *backbone* EfficientNet-B0 + capa de convolución separable en profundidad.
2. **Red de lenguaje:** BERT + BiGRU bidireccional.
3. **Componente de coincidencia:** espacio de embedding conjunto + pérdidas.

3.2.1 Red visual

Detector R-MHParsNet (no implementado en este proyecto). R-MHParsNet es una FPN (Feature Pyramid Network) basada en EfficientNet-B0, con cabezas de segmentación para generar máscaras de instancia humana. Genera los bounding boxes $\{(x_1, y_1, x_2, y_2)\}_{i=1}^n$ que delimitan cada persona detectada. Tiene 6,49M parámetros (un 79,9% más pequeño que su predecesor MHParsNet).

Generador de embedding visual (sí implementado). Para cada persona detectada, recortada como $\mathbf{I} \in \mathbb{R}^{384 \times 128 \times 3}$:

1. Se redimensiona a $\mathbb{R}^{3 \times 384 \times 128}$ (HxW o WxH según convención; el paper dice 384 alto $\times$ 128 ancho).
2. Pasa por EfficientNet-B0 y se extrae la salida de la **etapa 9** (la última): un tensor de $\mathbb{R}^{1280 \times 12 \times 4}$. Los 1280 canales son la profundidad de features; 12×4 son las dimensiones espaciales (es decir, $384/32 = 12$ y $128/32 = 4$).
3. Una capa de convolución separable en profundidad (DSC) con 1024 filtros 3×3 reduce a $\mathbb{R}^{1024 \times 12 \times 4}$.

Nota técnica: en este proyecto las imágenes entran a 512×512 (ver datasets/bases.py:43 y Capítulo 5), no a 384×128 . Eso cambia las dimensiones espaciales en el resto del pipeline. Se conserva la misma profundidad de canales.

3.2.2 Red de lenguaje

El componente de lenguaje procesa la descripción textual $\mathbf{T}$:

1. **BERT** tokeniza y recorta/padea a una longitud fija de 100 tokens: $(z_{+1}, \dots, z_{+100})$. El tokenizer es `bert-base-uncased`.
2. Una **capa de embedding** de 512 dimensiones convierte cada token en un vector denso: $(\mathbf{e}_1, \dots, \mathbf{e}_{100}) \in \mathbb{R}^{100 \times 512}$.
3. Una **GRU bidireccional** con 1024 unidades ocultas procesa la secuencia y produce:

$$\mathbf{T}' = (\overrightarrow{\text{GRU}} + \overleftarrow{\text{GRU}})/2 \in \mathbb{R}^{1024 \times 100 \times 1} \quad (3.4)$$

La suma promedio de las direcciones hacia adelante y hacia atrás captura el contexto en ambos sentidos.

3.2.3 Componente de coincidencia de embedding conjunto

Las dos modalidades se proyectan a un **espacio conjunto** de dimensión 1024:

- $\mathbf{I}' \in \mathbb{R}^{1024 \times 12 \times 4}$ (visual).
- $\mathbf{T}' \in \mathbb{R}^{1024 \times 100 \times 1}$ (textual).

Para optimizar el *matching* se usan dos pérdidas:

- **Pérdida de identidad** ($\mathcal{L}_{\text{ID}}$): cross-entropy que refuerza que cada persona tenga una representación única en el espacio.
- **Pérdida de ranking/clasificación** ($\mathcal{L}_{\text{rank}}$): basada en triplet loss, garantiza que las coincidencias correctas obtengan puntuaciones más altas que las incorrectas.

La suma ponderada es la pérdida total:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{rank}} + \beta \cdot \mathcal{L}_{\text{ID}} \quad (3.5)$$

En este proyecto $\alpha = \beta = 1,0$ (config.py:87-88).

3.3 Diagrama de bloques

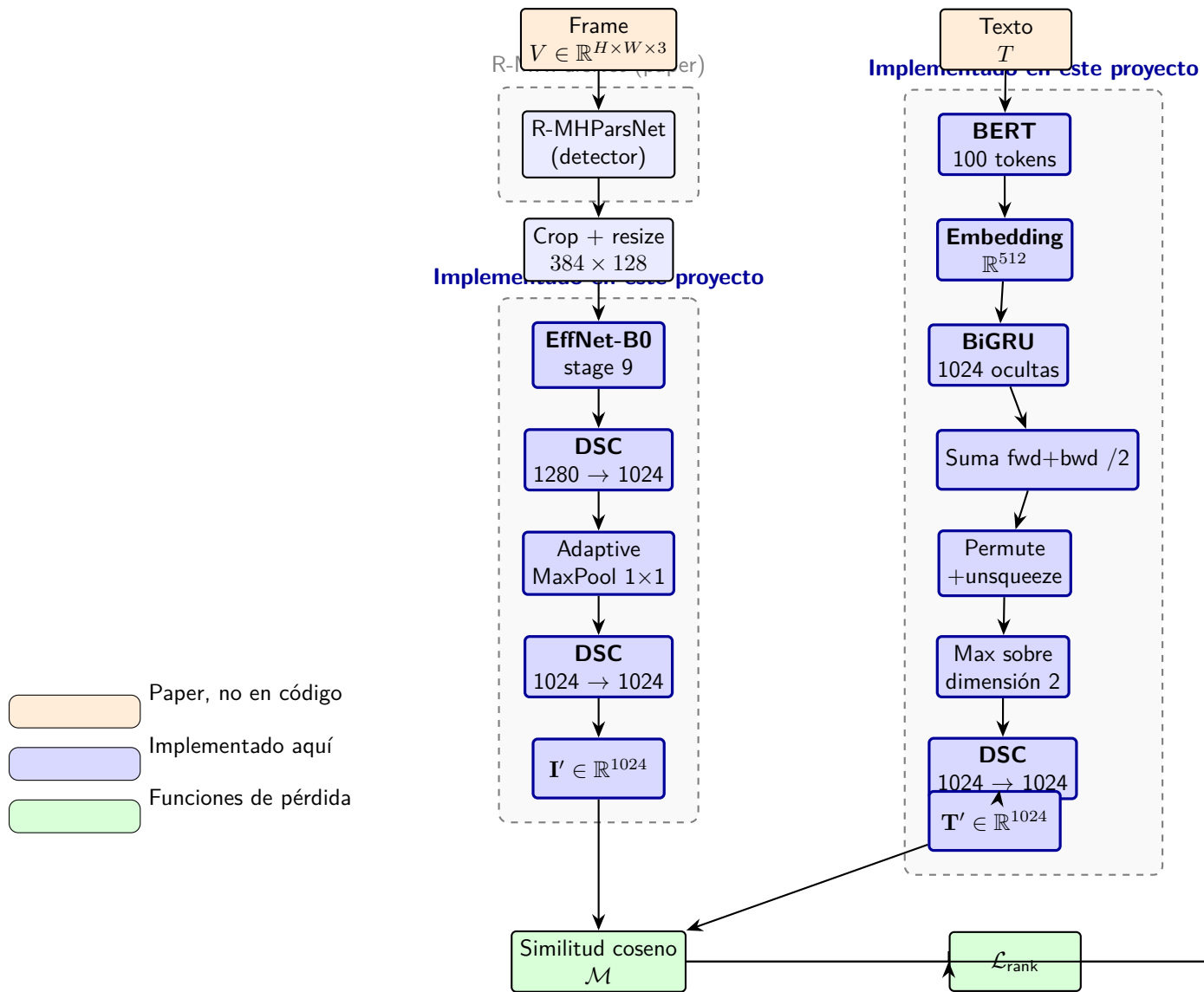


Figura 3.1: Arquitectura de U-TextReIDNet. Las cajas con relleno gris son las que este proyecto implementa; las cajas con relleno claro son las del paper que no están en el código (R-MHParsNet y sistema distribuido).

3.4 Dimensiones en cada etapa: tabla resumen

Tabla 3.1: Dimensiones de los tensores en cada etapa del modelo. B denota el tamaño de batch. Las columnas “Etapa paper” y “Etapa proyecto” difieren únicamente en la resolución espacial de entrada.

Componente	Forma	Origen	Ubicación
Imagen de entrada	$((B, 3, 512, 512))$	resize/colocar en canvas	datasets/bases.py: 128
EfficientNet-B0 stage 9	$((B, 1280, 16, 16))$	salida <code>model.features</code>	model/visual_network.py: 29
DSC 1280 $\rightarrow$ 1024	$((B, 1024, 16, 16))$	conv. separable en profundidad	model/textreidnet.py: 41
AdaptiveMaxPool2d(1,1)	$((B, 1024, 1, 1))$	pool global	model/textreidnet.py: 42
DSC 1024 $\rightarrow$ 1024	$((B, 1024, 1, 1))$	conv. separable en profundidad	model/textreidnet.py: 43
Visual embedding final	$((B, 1024, 1))$	<code>squeeze(-1)</code>	model/textreidnet.py: 71
Tokens de texto (BERT)	$((B, 100))$	<code>bert-base-uncased</code> + padding	datasets/bert_tokenizer.py: 17
Embedding	$((B, 100, 512))$	<code>nn.Embedding(29610, 512)</code>	model/language_network.py: 22
BiGRU	$((B, 100, 2048))$	bidir, 2×1024 ocultas	model/language_network.py: 24
Suma fwd + bwd / 2	$((B, 100, 1024))$	reduce a 1024 canales	model/language_network.py: 68
Permute + unsqueeze	$((B, 1024, 100, 1))$	prepara para DSC	model/language_network.py: 69
Max sobre dim 2	$((B, 1024, 1, 1))$	pool temporal	model/textreidnet.py: 87
DSC 1024 $\rightarrow$ 1024	$((B, 1024, 1, 1))$	conv. separable en profundidad	model/textreidnet.py: 88
Textual embedding final	$((B, 1024, 1))$	<code>squeeze(-1)</code>	model/textreidnet.py: 88

3.5 Por qué estas elecciones

¿Por qué EfficientNet-B0?

EfficientNet-B0 [?] introduce el concepto de *compound scaling*: en vez de escalar arbitrariamente profundidad, ancho o resolución, los escala simultáneamente con un coeficiente fijo derivado de una búsqueda automática. El resultado es un modelo con excelente relación accuracy/computo. La variante B0 es la más pequeña de la familia ($\sim 5,3$ M parámetros) y es suficiente para la tarea de extracción de features visuales.

¿Por qué BiGRU en lugar de LSTM o Transformer?

El paper justifica el uso de GRU por su eficiencia computacional: tiene menos parámetros que LSTM (3 compuertas vs 4) y es más rápido que un Transformer para secuencias cortas (100 tokens). La bidireccionalidad le permite capturar el contexto anterior y posterior, lo que mejora la comprensión de la descripción.

¿Por qué convolución separable en profundidad (DSC)?

Una convolución separable en profundidad [?] factoriza una convolución estándar en dos operaciones:

1. **Depthwise**: un filtro $k \times k$ por canal de entrada.
2. **Pointwise**: una convolución 1×1 para mezclar canales.

Reduce drásticamente los parámetros y las multiplicaciones, con una pérdida de accuracy mínima. En este proyecto se usa *dos* veces: para reducir $1280 \rightarrow 1024$ canales visuales y para proyectar al espacio conjunto.

¿Por qué 1024 como dimensión de embedding?

1024 es lo suficientemente grande para capturar la semántica de las dos modalidades, pero suficientemente pequeño para que la similitud del coseno sea estable y el entrenamiento sea rápido. Coincide con el número de unidades ocultas de la BiGRU.

3.6 Lo que cambia en este proyecto

El proyecto `textreid-train` implementa únicamente el camino de identificación (*TextReIDNet*) sin el detector R-MHParsNet. Esto significa:

- No se detecta automáticamente a las personas en imágenes con múltiples personas: la entrada debe ser una imagen con una sola persona (post-recortada o de datasets tipo CUHK-PEDES).
- Se conserva la *colocación sobre canvas negro* (`datasets/bases.py:43`) porque el paper entrena así (las imágenes de CUHK-PEDES son menores que 512×512 y se colocan aleatoriamente sobre un lienzo negro para preservar aspect ratio).
- El tamaño de canvas es 512×512 en el código (no 384×128 como en el paper). Esto es un punto a tener en cuenta si se quiere reproducir exactamente los números del paper.

Estos detalles se discuten con más profundidad en los capítulos siguientes.

Capítulo 4

Sistema descentralizado multicámara

Este capítulo describe el sistema completo de vigilancia en el que se enmarca U-TextReIDNet. **No hay código en este proyecto que lo implemente**, pero es útil entenderlo para saber qué parte del paper es la que realmente vive en `textreid-train`.

4.1 Arquitectura del sistema

El sistema propuesto en `docs/paper.md` (§III-B) tiene tres capas:

1. **Nodos de cámara** ($d_1, \dots, d_n$): capturan vídeo, mantienen galerías locales y ejecutan U-TextReIDNet para hacer inferencia local.
2. **Infraestructura de comunicaciones**: red entre nodos y estación de control.
3. **Estación de control**: servidor de aplicaciones (con U-TextReIDNet también) + aplicación web (*Flask*).

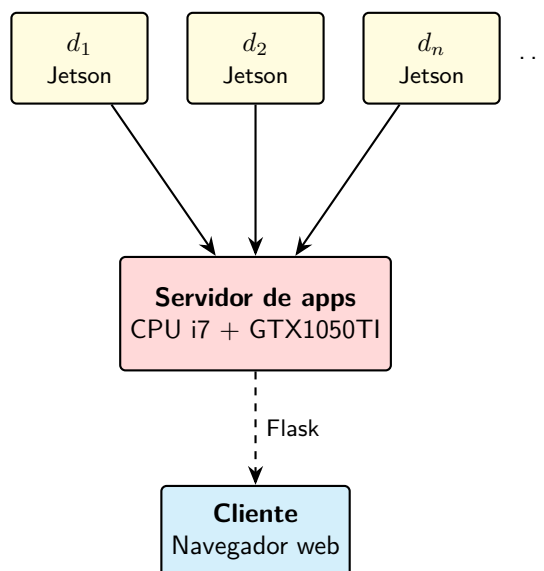


Figura 4.1: Topología del sistema descentralizado. Cada cámara corre U-TextReIDNet localmente; sólo transmite vídeo cuando su puntuación supera un umbral y es la mejor de todas.

4.2 Flujo operativo

El paper describe cinco fases (`docs/paper.md`, líneas 185–195):

a) Inicialización y solicitud

Al encenderse, cada cámara se conecta al servidor de aplicaciones y reporta su estado (*online-idle*, *online-búsqueda*, *transmitiendo*, *offline*). El usuario inicia o detiene búsquedas desde la aplicación.

b) Captura de fotogramas y galería local

Cuando una cámara recibe la orden de búsqueda, captura vídeo y usa R-MHParsNet para generar una galería local de instancias humanas detectadas en el fotograma actual. *Cada fotograma se trata como datos nuevos*: no hay relación temporal entre galerías consecutivas.

c) Inferencia local

Cada cámara calcula la similitud coseno entre cada imagen de su galería local y la descripción textual proporcionada por el usuario. Selecciona la imagen mejor clasificada y, si su puntuación supera un umbral θ , la procesa para incluir un bounding box alrededor de la persona objetivo.

d) Identificación global

Cada segundo, todas las cámaras cuyas puntuaciones superen θ envían sus puntuaciones al servidor. El servidor selecciona la puntuación más alta y notifica a esa cámara para que transmita vídeo.

e) Finalización

El usuario debe detener una búsqueda antes de iniciar otra.

4.3 Implementación de la estación de control

La aplicación de usuario es una webapp *Flask* [?] que corre en el servidor de aplicaciones. Se divide en dos subsistemas:

- **Subsistema de comunicación:** usa **ZMQ** para mensajería asíncrona entre el servidor y los nodos, y **MQTT** para publicar resultados de búsqueda desde los nodos al servidor.
- **Subsistema de vídeo:** usa **ImageZMQ** para gestionar las transmisiones de vídeo entrantes cuando se detecta a la persona objetivo.

Estos subsistemas corren en hilos separados para que la UI no se bloquee durante el procesamiento.

4.4 Por qué este proyecto no incluye el sistema

El repositorio `textreid-train` es una versión *training-only* del proyecto original. El README lo explica:

El proyecto original es un sistema completo de vigilancia multicámara descentralizada con scripts de inferencia, una webapp Flask, MySQL/MQTT y detección humana. *Esta versión quita todo eso* – sólo tiene lo necesario para **entrenar** y **evaluar**.

Las piezas eliminadas son:

- `model/unity.py`: wrapper de detección + re-ID (innecesario para entrenamiento puro).
- `model/human_detection_network.py`: detector (R-MHParsNet no se entrena aquí).
- `model/feature_pyramid_network.py`: usado solo por el detector.
- `model/mask_head.py`, `parsing_head.py`: cabezas del detector.

- `surveillance_application/`: la webapp Flask.
- `demo/`: demo web.
- `inference/`: scripts de inferencia sobre imágenes/vídeos.

Para desplegar el sistema completo habría que integrar este modelo con un detector (YOLOv8, por ejemplo) y la infraestructura descrita arriba. Eso queda fuera del alcance del libro.

4.5 Rendimiento reportado en el paper

El paper mide tiempos de inferencia en dos plataformas:

Tabla 4.1: Tiempos de inferencia reportados en el paper (media sobre 20 iteraciones).

Modelo	PC (CPU+GPU)	Jetson Nano
MFPE	0,69204 s	—
TextReIDNet	0,00477 s	—
U-TextReIDNet	0,04995 s	0,75013 s

Para una galería de 10 personas:

Tabla 4.2: Tiempos de inferencia para 10 imágenes de galería.

Modelo	PC	Jetson Nano
MFPE	2,46102 s	—
TextReIDNet	1,28512 s	—
U-TextReIDNet	0,09525 s	0,75013 s

La conclusión clave es que U-TextReIDNet puede procesar galerías pequeñas en tiempo real incluso en hardware embebido, y su tiempo de inferencia escala linealmente con el número de personas ($O(N \cdot n)$ donde N es la densidad y n el número de características).

4.6 Lo que este proyecto sí hace

Aunque no incluye el sistema distribuido, el modelo entrenado en `textreid-train` es el mismo TextReIDNet que se usa en el paper como submódulo de U-TextReIDNet. Por tanto, un checkpoint producido aquí puede integrarse en el sistema completo con sólo cambiar la arquitectura de detección.

Capítulo 5

El dataset CUHK-PEDES

Este capítulo describe el dataset con el que se entrena y evalúa el modelo: qué contiene, qué formato tiene en disco, cómo lo carga el proyecto y qué decisiones de preprocesamiento se aplican.

5.1 El dataset en cifras

CUHK-PEDES (*Person Description in the Wild*) [?] es el benchmark estándar para re-ID por texto. La versión *original* tiene 40 206 imágenes y 80 440 descripciones (2 por imagen). Este proyecto soporta dos variantes:

Tabla 5.1: Diferencias entre la versión original y la versión HuggingFace de CUHK-PEDES.

	Original (paper)	HuggingFace
Total imágenes	40 206	34 042 (este proyecto)
Captions por imagen	2	~7 (1 por fila)
Identidades (pids)	13 003	derivadas del hash de imagen
Splits	train / val / test	derivados: 75 % / 12,5 % / 12,5 %
Resolución	variable (~256×128+)	128×128 thumbnails
Acceso	Email a Tong Xiao	Inmediato desde HuggingFace
Top-1 esperado	~54 % (paper)	~45-52 %

Recomendación: si se pueden obtener los datos originales por email (gratis para investigación), se obtienen mejores números. Si no, la versión HF funciona perfectamente para validar el pipeline.

5.2 Formato en disco

Después de ejecutar `scripts/download_hf_dataset.py`, la versión HF queda en `data/CUHK-PEDES-HF/` con esta estructura:

```
data/CUHK-PEDES-HF/
|---- raw.parquet          # snapshot de HuggingFace (550 MB)
|---- reid_raw.json        # anotaciones en formato "estándar"
|-- imgs/
    |-- imgs/all/
        |-- 7138159738431296927.jpg
        |-- 3825926175112857003.jpg
        `-- ... (34042 imágenes en total)
```

5.2.1 El archivo reid_raw.json

Es la *fuentes de verdad* del dataset. Cada elemento es un diccionario con estos campos:

```

1  {
2    "file_path": "all/7138159738431296927.jpg",
3    "captions": [
4      "A pedestrian with dark hair is wearing red and white shoes, "
5      "a black hooded sweatshirt, and black pants.",
6      "The person has short black hair and is wearing black pants, "
7      "a long sleeve black top, and red sneakers.",
8      ...
9    ],
10   "split": "train",
11   "processed_tokens": [],
12   "id": 0
13 }
```

Los campos obligatorios son sólo `file_path` y `captions`. Los demás se completan en tiempo de carga.

5.3 Cómo se cargan los datos

El proyecto tiene dos clases de carga según la fuente:

- `datasets/cuhkpedes.py`: CUHKPEDES para datos originales.
- `datasets/cuhkpedes_hf.py`: CUHKPEDESHF para datos de HuggingFace.

Ambas clases exponen la misma interfaz:

```

1  ds.train          # lista de (pid, image_id, img_path, caption)
2  ds.train_id_container # set de pids en train
3  ds.val            # dict con image_pids, img_paths, caption_pids, captions
4  ds.test           # dict idem para test
```

5.3.1 El loader HF: CUHKPEDESHF

Esta clase (`datasets/cuhkpedes_hf.py`) tiene tres pasos clave:

1. **Asignación de pids**: como la versión HF no incluye IDs de persona, los deriva del *hash de la imagen*. Las imágenes idénticas (mismas captions) comparten el mismo pid.
2. **División train/val/test**: por proporción de IDs: 75 % / 12,5 % / 12,5 %. No usa el campo `split` del JSON.
3. **Procesamiento de anotaciones**: igual que la versión original.

El código clave está en `datasets/cuhkpedes_hf.py`:**62-71**:

```

def _build_pid_mapping(self):
    """Group entries by image_path (which is the identity key)."""
    self.path_to_id = {}
    next_id = 0
    for anno in self.annos:
        fp = anno['file_path']
        if fp not in self.path_to_id:
            self.path_to_id[fp] = next_id
            next_id += 1
    self.num_persons = next_id
```

5.3.2 El loader original: CUHKPEDES

La clase `datasets/cuhkpedes.py` asume que las anotaciones vienen con `split` explícito (train/val/test) y con `id` entre 1 y 13003. Internamente resta 1 al `id` para que empiece en 0 (más conveniente para PyTorch). El código relevante está en `datasets/cuhkpedes.py`:**75-115**.

5.4 El dataloader unificado

`datasets/cuhkpedes_dataloader.py` es el punto de entrada que decide qué loader usar y construye los `DataLoaders` de PyTorch. Su función principal es `build_cuhkpedes_dataloader(config)`.

```
def build_cuhkpedes_dataloader(config: dict = None):
    if config.dataset_source == 'huggingface':
        dataset_object = CUHKPEDESHF(config)
    else:
        dataset_object = CUHKPEDES(config)

    train_transform = T.Compose([
        T.ToTensor(),
        T.Normalize(mean=config.MHPV2_means, std=config.MHPV2_stds)
    ])
    inference_transform = T.Compose([
        T.ToTensor(),
        T.Normalize(mean=config.MHPV2_means, std=config.MHPV2_stds)
    ])
    train_num_classes = len(dataset_object.train_id_container)

    train_set = ImageTextDataset(
        dataset_object.train,
        train_transform,
        tokenizer_type=config.tokenizer_type,
        tokens_length_max=config.tokens_length_max
    )

    train_data_loader = DataLoader(
        train_set,
        batch_size=config.batch_size,
        shuffle=True,
        num_workers=config.num_workers,
        collate_fn=collate
    )
    ...
    return train_data_loader, inference_img_loader,
        inference_txt_loader, train_num_classes
```

Nota importante: las transformaciones de imagen *sólo* convierten a tensor y normalizan. El `resize` a 384×128 que el paper describe **no está**. Las imágenes se quedan en su tamaño nativo (512×512 después del `place_image_on_canvas`).

5.5 La clase ImageTextDataset

Es el `Dataset` de PyTorch que produce un batch mixto imagen+texto. Vive en `datasets/bases.py`. Para cada índice:

```
def __getitem__(self, index):
    pid, image_id, img_path, caption = self.dataset[index]
```

```

pid = torch.from_numpy(np.array([pid])).long()
img = read_image(img_path)
img = place_image_on_canvas(img)
original_img = img.copy()
if self.transform is not None:
    img = self.transform(img)
# ... guarda también la imagen original sin normalizar
tokens = self.tokenizer(caption)
token_ids, orig_token_length = pad_tokens(tokens,
                                           self.tokens_length_max)

ret = {'pids': pid,
       'image_ids': image_id,
       'img_paths': img_path,
       'preprocessed_images': img,
       'original_images': original_img,
       'token_ids': token_ids.to(torch.long),
       'orig_token_lengths': orig_token_length,
       'captions': caption}
return ret

```

Cada ítem del dataset devuelve un diccionario con 8 claves. El `collate` (`utils/miscellaneous_utils.py:189`) se encarga de apilar tensores y dejar las cadenas como listas.

5.6 Preprocesamiento de imagen: `place_image_on_canvas`

Función definida en `datasets/bases.py`:

```

def place_image_on_canvas(image, new_size=(512, 512),
                          placement_strategy='random'):
    """Plaza una imagen en un canvas negro preservando aspect ratio."""
    if image.height > 512:
        aspect_ratio = image.width / image.height
        new_height, _ = new_size
        new_width = int(aspect_ratio * new_height)
        image = image.resize((new_width, new_height), Image.ANTIALIAS)

    canvas = Image.new('RGB', new_size, (0, 0, 0))

    if placement_strategy == 'center':
        x = (new_size[0] - image.width) // 2
        y = (new_size[1] - image.height) // 2
    elif placement_strategy == 'random':
        x = np.random.randint(0, max(1, new_size[0] - image.width))
        y = np.random.randint(0, max(1, new_size[1] - image.height))
    else:
        raise ValueError(...)

    canvas.paste(image, (x, y))
    return canvas

```

¿Por qué? Las imágenes de CUHK-PEDES son chicas ($\sim 128 \times 128$ en HF, $\sim 256 \times 128$ en original). EfficientNet-B0 espera imágenes relativamente grandes (224×224 o más). Para no distorsionar el aspect ratio, se pega la imagen en una posición aleatoria sobre un canvas negro de 512×512 . Esto introduce variación posicional como augmentation implícita.

5.7 Preprocesamiento de texto

Tokenización

Hay tres tokenizers disponibles; el que se usa por defecto es **BERT** (`bert-base-uncased`).

```
class BERTTokenizer(object):
    def __init__(self):
        super(BERTTokenizer, self).__init__()
        _ , tokenizer_class, pretrained_weights = (
            ppb.BertModel, ppb.BertTokenizer, 'bert-base-uncased')
        self.tokenizer = tokenizer_class.from_pretrained(
            pretrained_weights)

    def __call__(self, text: str = None) -> torch.LongTensor:
        tokens = self.tokenizer.encode(text)
        return tokens
```

Otros tokenizers disponibles en `datasets/simple_tokenizer.py` y `datasets/tiktoken_tokenizer.py`, pero el config (`config.py:41`) fija `tokenizer_type='bert'` por defecto.

Padding

La función `pad_tokens` (`utils/miscellaneous_utils.py:86-107`) recorta o padea a longitud fija:

```
def pad_tokens(tokens, tokens_length_max):
    tokens_length = len(tokens)
    tokens = torch.from_numpy(np.array(tokens)).view(-1)
        .contiguous().float()

    if tokens_length < tokens_length_max:
        zero_padding = torch.zeros(tokens_length_max - tokens_length)
        tokens = torch.cat([tokens, zero_padding], 0)
    else:
        tokens = tokens[:tokens_length_max]
        tokens_length = tokens_length_max
    return tokens, tokens_length
```

Tokens de padding = 0. Esto es importante porque el `nn.Embedding` del modelo tiene `padding_idx=0` (`model/language_network.py:22`), lo que garantiza que el padding no afecte al embedding.

5.8 Normalización de imagen

El dataloader aplica `T.Normalize` con medias y desviaciones de MHPV2 (no de ImageNet):

1	<code>mean = [0.3555248975753784, 0.3295726776123047, 0.3115333914756775]</code>
2	<code>std = [0.3482806384563446, 0.3339986503124237, 0.3296058773994446]</code>

definidos en `config.py:50-51`. **Esto es un poco raro:** el paper usaría medias de ImageNet (`[0,485, 0,456, 0,406]`) que son las guardadas en `CUHKPEDES_means` (`config.py:52`). El código actual usa MHPV2 (que es *otro* dataset). Es un detalle a tener en cuenta al reproducir resultados.

5.9 El cuaderno de validación

En `notebooks/01\_lab\_validate\_dataset.ipynb` hay un notebook de Jupyter que verifica:

1. Que el JSON existe y tiene los campos correctos.
2. Que las imágenes referenciadas existen y son válidas.
3. La distribución de longitud de captions.
4. La distribución de splits.
5. Que un forward pass del modelo funciona.
6. Las pérdidas dan valores finitos.
7. Una estimación de VRAM para elegir el `batch_size`.

Conviene correr este notebook *antes* del primer entrenamiento, especialmente si se usa una GPU diferente a la del paper.

5.10 Dataset augmentation

Este proyecto **no aplica augmentation** explícita de imagen más allá de:

- Posición aleatoria en el canvas (`placement_strategy='random'`).
- Normalización con medias MHPV2.

El paper original describe un pipeline más rico (docs/paper.md:245): *redimensionamiento, recorte aleatorio y normalización*. Este proyecto lo omite; podría añadirse en `get_transform` (utils/miscellaneous_utils.py:210).

5.11 Resumen del pipeline de datos

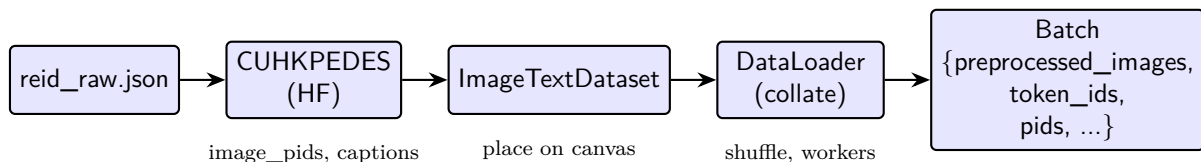


Figura 5.1: Pipeline de datos desde el JSON hasta el batch que consume `train.py`.

Capítulo 6

Configuración del proyecto

Todo el comportamiento del proyecto está controlado por `config.py`. Este archivo define un diccionario de configuración (`DotDict`) que se pasa a cada módulo y permite modificar el experimento sin tocar el código.

6.1 La clase `DotDict`

```
class DotDict(dict):
    __getattr__ = dict.get
    __setattr__ = dict.__setitem__
    __delattr__ = dict.__delitem__
```

Es un `dict` con acceso por atributo: `configs.lr` es equivalente a `configs['lr']`. Esto permite usar tanto `config.feature_length` como `config['feature_length']` en el resto del código.

6.2 Función principal: `sys_configuration`

Es la única función exportada. Acepta tres parámetros y devuelve un `DotDict`:

```
def sys_configuration(platform_name: str = platform.node(),
                      dataset_name: str = "CUHK-PEDES",
                      dataset_source: str = "huggingface",
                      show_configs: bool = False) -> DotDict:
    """
    Args.:
        platform_name: hostname of the machine
        dataset_name: "CUHK-PEDES" (only one supported for now)
        dataset_source: "huggingface" or "original"
    """
```

Los argumentos se pueden sobrescribir desde CLI en `train.py` (`train.py:56`).

6.3 Tabla maestra de parámetros

Modelo de lenguaje

Parámetro	Valor	Efecto / ubicación
dropout_rate	0.30	Dropout tras el embedding textual. config.py: 37 , model/language_network.py: 23 .
num_layers	1	Número de capas del GRU bidireccional. config.py: 38 , model/language_network.py: 24 .
tokens_length_max	100	Longitud fija (con padding) de cada caption tokenizada. config.py: 39 , datasets/bases.py: 96 .
feature_length	1024	Dimensión del espacio de embedding conjunto. config.py: 40 , model/textreidnet.py: 43 .
tokenizer_type	'bert'	Tipo de tokenizer. Otros: 'simple_tokenizer', 'tiktoken_*'. config.py: 41 , datasets/bases.py: 97 .
vocab_size	29 610	29 609 (BERT) + 1 (padding). Padding idx = 0. config.py: 42 , model/language_network.py: 22 .
embedding_dim	512	Dimensión del embedding textual. config.py: 43 , model/language_network.py: 22 .

Tabla 6.1: Parámetros del modelo de lenguaje.

Modelo visual

Parámetro	Valor	Efecto / ubicación
CUHKPEDES_image_size	(384, 128)	Tamaño <i>declarado</i> en el paper. No se usa en el dataloader actual. config.py: 48 .
MHPV2_image_size	(512, 512)	Tamaño del canvas en el que se pegan las imágenes. config.py: 49 , datasets/bases.py: 58 .
MHPV2_means	[0.355, 0.330, 0.311]	Medias para T.Normalize en el dataloader. config.py: 50 .
MHPV2_stds	[0.348, 0.334, 0.330]	Desviaciones para T.Normalize. config.py: 51 .
CUHKPEDES_means	[0.485, 0.456, 0.406]	Medias de ImageNet (no usadas en este proyecto). config.py: 52 .
CUHKPEDES_stds	[0.229, 0.224, 0.225]	Desviaciones de ImageNet (no usadas). config.py: 53 .

Tabla 6.2: Parámetros del modelo visual.

Entrenamiento

Parámetro	Valor	Efecto / ubicación
epoch	60	Número de epochs total. config.py: 71 , train.py: 113 .
adam_alpha	0.90	β_1 de AdamW. config.py: 72 , train.py: 98 .
adam_beta	0.999	β_2 de AdamW. config.py: 73 , train.py: 98 .
epoch_decay	[20, 40]	Épocas donde el LR scheduler decae el learning rate. config.py: 74 , train.py: 99 .
lr	0.001	Learning rate inicial. config.py: 75 , train.py: 98 .
patience	3	(No usado actualmente; sería early stopping). config.py: 76 .
val_dataset	'test'	Split usado durante validación. config.py: 77 .
seed	3407	Semilla para reproducibilidad. config.py: 79 , train.py: 71 .
batch_size	8 (default)	Tamaño de batch (4 en Jetson). config.py: 123,126,129 .
num_workers	4 (default)	Hilos del DataLoader. config.py: 122,125,128 .

Tabla 6.3: Parámetros de entrenamiento.

Pérdidas

Parámetro	Valor	Efecto / ubicación
margin	0.5	Margen del triplet-style RankingLoss. config.py: 86 , evaluation/ranking_loss.py: 41 .
ranking_loss_alpha	1.0	Peso α de la RankingLoss en la pérdida total. config.py: 87 , train.py: 145 .
identity_loss_beta	1.0	Peso β de la IdentityLoss en la pérdida total. config.py: 88 , train.py: 146 .

Tabla 6.4: Parámetros de funciones de pérdida.

Paths y logging

Parámetro	Valor	Ubicación
train_log_path	logs/train.log	config.py:93
test_log_path	logs/test.log	config.py:94
model_save_path	data/checkpoints	config.py:96
plot_save_path	data/plots	config.py:97
dataset_path (HF)	data/CUHK-PEDES-HF	config.py:108
dataset_path (orig)	data/CUHK-PEDES	config.py:112

Tabla 6.5: Paths del proyecto.

6.4 Override por línea de comandos

train.py permite sobrescribir los parámetros más importantes:

```

1 python train.py \
2   --dataset_source huggingface \
3   --epochs 60 \
4   --batch_size 8 \
5   --lr 0.001 \
6   --seed 3407 \
7   --output_dir ./data/checkpoints

```

Cada flag corresponde a un campo del config que se sobrescribe justo después de llamar a sys_configuration (train.py:56-69).

6.5 Configuración por plataforma

Hay una rama de detección automática de plataforma basada en el hostname (config.py:121-129):

```

if platform_name in ('PC-SIM', 'deeplearning', 'ultron', 'yrsn'):
    configs['num_workers'] = 4
    configs['batch_size'] = 8
elif platform_name == 'nano':
    configs['num_workers'] = 2
    configs['batch_size'] = 4
else:
    configs['num_workers'] = 4
    configs['batch_size'] = 8

```

Si tu hostname es “nano” (Jetson Nano), el batch se reduce automáticamente a 4. Para cualquier otro caso, usa 8.

6.6 Reproducibilidad

set_seed (utils/miscellaneous_utils.py:18-30) fija:

- PYTHONHASHSEED.
- random.seed, numpy.random.seed.
- torch.manual_seed y torch.cuda.manual_seed.
- cudnn.deterministic=True, cudnn.benchmark=False.

Esto garantiza resultados idénticos entre ejecuciones en la misma máquina. No garantiza resultados idénticos entre máquinas distintas.

6.7 Ejemplo de inspección

Para imprimir la configuración completa desde un script:

```
1 from config import sys_configuration
2 cfg = sys_configuration(dataset_source='huggingface')
3 print(cfg.feature_length)      # 1024
4 print(cfg.batch_size)          # 8
5 print(cfg.vocab_size)          # 29610
```

Esta es la mejor forma de verificar que la configuración es la esperada antes de entrenar.

Capítulo 7

Red visual: EfficientNet-B0 y convolución separable

Este capítulo combina **explicaciones conceptuales** de las piezas usadas en la rama visual con el **código línea por línea** del proyecto.

7.1 ¿Qué es EfficientNet?

7.1.1 El problema

Hasta 2019, la mayoría de las CNNs se escalaban de forma *arbitraria*: o se hacían más profundas (más capas), o más anchas (más canales), o con mayor resolución de entrada. Cada familia tenía su propia receta y nadie sabía cuál era la mejor.

7.1.2 Compound scaling

EfficientNet-B0 [?] introdujo una idea elegante: **escalar las tres dimensiones simultáneamente** con un ratio fijo (ϕ). El paper demostró que esto da mejor accuracy por FLOP que cualquier escalado aislado.

$$\begin{aligned}\text{depth} &= \alpha^\phi \\ \text{width} &= \beta^\phi \\ \text{resolution} &= \gamma^\phi \\ \text{con } \alpha \cdot \beta^2 \cdot \gamma^2 &\approx 2\end{aligned}\tag{7.1}$$

La familia EfficientNet va desde B0 (la más chica) hasta B7 (la más grande). En este proyecto se usa **B0** porque es la única que cabe en hardware modesto con un budget razonable.

7.1.3 ¿Qué es MBConv?

MBConv (*Mobile Inverted Bottleneck Convolution*) es el bloque básico de EfficientNet. Inventado por MobileNetV2 y mejorado por EfficientNet. Tiene tres etapas:

1. **Expansión**: conv 1x1 que aumenta canales ($\times 6$ típicamente, por eso "MBConv6").
2. **Depthwise conv**: conv $k \times k$ por canal (máscara espacial).
3. **Proyección**: conv 1x1 que reduce canales.

A diferencia del bottleneck clásico de ResNet, MBConv *expande* primero y *comprime* después (inverted).

7.1.4 Layout de B0

Tabla 7.1: Las 9 etapas de EfficientNet-B0.

Stage	Operador	Resolución	Canales	Layers
1	Conv3x3	224×224	32	1
2	MBCConv1, k3x3	112×112	16	1
3	MBCConv6, k3x3	112×112	24	2
4	MBCConv6, k5x5	56×56	40	2
5	MBCConv6, k3x3	28×28	80	3
6	MBCConv6, k5x5	14×14	112	3
7	MBCConv6, k5x5	14×14	192	4
8	MBCConv6, k3x3	7×7	320	1
9	Conv1x1+Pool	7×7	1280	1

Cada etapa divide la resolución a la mitad (excepto 2 y 5 que mantienen). La **etapa 9** es la que se usa en TextReIDNet: produce 1280 canales a 7×7 (con entrada 224×224) o 16×16 (con entrada 512×512).

7.2 ¿Qué es la convolución separable en profundidad (DSC)?

7.2.1 El problema

Una convolución 2D estándar $k \times k$ sobre una imagen con C canales de entrada y C' canales de salida tiene:

- **Parámetros:** $k \cdot k \cdot C \cdot C'$.
- **FLOPs:** $k^2 \cdot C \cdot C' \cdot H \cdot W$.

Para $k = 3$, $C = C' = 1280$: **14,7M** parámetros y muchos FLOPs.

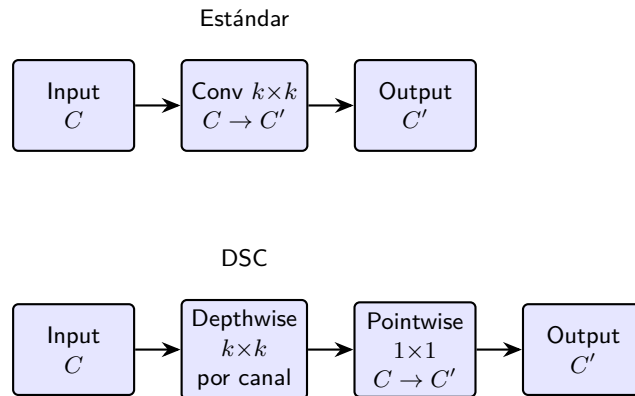
7.2.2 La factorización

La *depthwise separable convolution* (DSC) [?] factoriza esa operación en dos:

1. **Depthwise:** una convolución $k \times k$ por canal, con $k^2 \cdot C$ parámetros (en vez de $k^2 \cdot C \cdot C'$).
2. **Pointwise:** una convolución 1×1 para mezclar canales, con $C \cdot C'$ parámetros.

Resultado: $k^2 \cdot C + C \cdot C'$ parámetros, una reducción de $\sim k^2$ veces. Para $k = 3$: $\sim 9\times$.

7.2.3 Visualización



$$\begin{aligned} \text{Parámetros: estándar} &= k^2 \cdot C \cdot C' \\ \text{DSC} &= k^2 \cdot C + C \cdot C' \\ &\quad (\text{reducción} \approx k^2 \text{ veces}) \end{aligned}$$

Figura 7.1: Comparación entre convolución estándar y DSC.

7.3 ¿Qué es AdaptiveMaxPool2d?

Un *max pooling adaptativo* recibe como argumento el tamaño *deseado de salida*, no el stride ni el kernel. Calcula automáticamente los hiperparámetros para que el tensor de salida tenga exactamente esa forma.

- `nn.AdaptiveMaxPool2d((1, 1))`: salida siempre $(B, C, 1, 1)$.
- Para cada canal c , devuelve el valor máximo sobre todas las posiciones espaciales.
- Equivale a un *GAP* (global max pooling) si el kernel efectivo cubre toda la imagen.

¿Por qué? Después del EfficientNet tenemos un tensor $(B, 1024, 16, 16)$. Necesitamos un vector por imagen. El max pooling global colapsa $(16, 16) \rightarrow (1, 1)$ quedándose con la activación más fuerte de cada canal.

7.4 Código: la clase VisualNetwork

```
class VisualNetwork(nn.Module):
    """Backbone visual: EfficientNet-B0 preentrenado."""

    def __init__(self):
        # Constructor de nn.Module (registra parámetros)
        super(VisualNetwork, self).__init__()

        # Cargar EfficientNet-B0 con pesos preentrenados en ImageNet
        # EfficientNet_B0_Weights.DEFAULT = mejores pesos disponibles
        # (~5.3M parámetros, input esperado 224x224)
        self.model = models.efficientnet_b0(
            weights=EfficientNet_B0_Weights.DEFAULT
        )

        # `model.features` es un nn.Sequential con 9 bloques
        # (ver Tabla 6.1). Vamos a iterar bloque por bloque.
```

```

self.feature_blocks = self.model.features

def forward(self, x):
    """x: (B, 3, H, W) imagen preprocesada.
    returns: (B, 1280, H/32, W/32) salida etapa 9."""
    # Lista para acumular features de cada bloque
    features = []

    # Iterar sobre los 9 bloques secuenciales de EfficientNet-B0
    for block in self.feature_blocks:
        # Aplicar bloque (conv + BN + activation)
        x = block(x)

        # Guardar feature de este bloque
        features.append(x)

    # Devolver SOLO la salida del último bloque (etapa 9)
    # = 1280 canales, resolución espacial /32
    return features[-1]

```

7.5 Código: DepthwiseSeparableConv

```

class DepthwiseSeparableConv(nn.Module):
    """Convolución separable en profundidad: depthwise + pointwise."""

    def __init__(self, in_channels=None, out_channels=None,
                  kernel_size=3, bias=True, is_final_conv=False):
        # Constructor de nn.Module
        super(DepthwiseSeparableConv, self).__init__()

        # Depthwise convolution: una convolución k×k por canal
        # groups=in_channels hace que cada filtro actúe sobre UN canal
        # padding=1 mantiene la dimensión espacial
        self.depthwise_convolution = nn.Conv2d(
            in_channels=in_channels,
            out_channels=in_channels,      # mismo número de canales
            kernel_size=kernel_size,
            groups=in_channels,             # ← clave: depthwise
            bias=bias,
            padding=1                       # mantiene H, W
        )

        # Pointwise convolution: 1×1 que mezcla canales
        # in_channels -> out_channels
        self.pointwise_convolution = nn.Conv2d(
            in_channels=in_channels,
            out_channels=out_channels,     # ← dimensión de salida
            kernel_size=1,
            groups=1,                       # convolución normal
            bias=bias
        )

        # Encadenar las dos operaciones
        self.depthwise_separable_conv = nn.Sequential(
            self.depthwise_convolution,
            self.pointwise_convolution
        )

        # RetinaNet-style bias init (no se usa en TextReIDNet)

```

```

self.is_final_conv = is_final_conv
if bias and is_final_conv:
    self.is_final_conv = True

# Inicialización Kaiming (para Conv2d y Linear)
self.depthwise_separable_conv.apply(weights_init_kaiming)

def forward(self, x):
    return self.depthwise_separable_conv(x)

```

7.6 Etapas con shapes exactas (input 512×512)

Tabla 7.2: Etapas con shapes para input 512×512 en este proyecto.

Stage	Operador	Output HxW	Output Ch	Layers
0	Conv3x3	256×256	32	1
1	MBCConv1, k3x3	256×256	16	1
2	MBCConv6, k3x3	128×128	24	2
3	MBCConv6, k5x5	64×64	40	2
4	MBCConv6, k3x3	32×32	80	3
5	MBCConv6, k5x5	16×16	112	3
6	MBCConv6, k5x5	16×16	192	4
7	MBCConv6, k3x3	8×8	320	1
8	Conv1x1	8×8	1280	1

7.7 Flujo de tensores

```

1  # Entrada: imagen preprocesada (normalizada)
2  x: (B, 3, 512, 512)
3
4  # EfficientNet-B0
5  x = visual_network(x)
6  # -> (B, 1280, 16, 16)  <- etapa 9, depende del tamaño de entrada
7
8  # DSC 1280 -> 1024
9  x = visual_features_downscale(x)
10 # -> (B, 1024, 16, 16)
11
12 # AdaptiveMaxPool2d((1, 1))
13 x = adaptive_max_pooling(x)
14 # -> (B, 1024, 1, 1)
15
16 # DSC final 1024 -> 1024
17 x = depthwise_seperable_convolution(x)
18 # -> (B, 1024, 1, 1)
19
20 # squeeze última dim
21 x = x.squeeze(-1).contiguous()
22 # -> (B, 1024, 1)

```

7.8 Parámetros

Tabla 7.3: Parámetros por componente visual.

Componente	Parámetros
EfficientNet-B0 features	~5,29 M
DSC(1280, 1024)	~14 K
AdaptiveMaxPool2d(1,1)	0
DSC(1024, 1024)	~10 K
Total visual	~5,31 M

Capítulo 8

Red de lenguaje: BERT, embeddings, BiGRU y pack-padded

Este capítulo explica los conceptos de NLP detrás de la rama textual del modelo y luego muestra el código línea por línea.

8.1 ¿Qué es BERT y por qué sólo su tokenizer?

8.1.1 BERT

BERT (*Bidirectional Encoder Representations from Transformers*) [?] es un modelo de lenguaje preentrenado en un corpus enorme de texto. Tiene dos partes:

1. **Tokenizer:** convierte texto a IDs numéricos usando un vocabulario fijo. Es lo único que usamos aquí.
2. **Transformer encoder:** 12 capas de atención que producen embeddings contextuales. **No lo usamos.**

8.1.2 ¿Por qué no usar BERT entero?

BERT completo tiene ~110M parámetros. BiGRU sólo ~4.7M. Para textos cortos (descripciones de 30 palabras) BiGRU captura suficiente contexto. Esto reduce:

- Coste computacional (~20× menos).
- VRAM (cabe en GPU modestas).
- Tiempo de entrenamiento.

8.1.3 Cómo funciona el tokenizer de BERT

BERT usa *WordPiece tokenization*: el vocabulario está formado por sub-palabras (*tokens*) frecuentes más letras individuales. Ejemplo:

```
Entrada: "the man is wearing a black jacket"
Tokens: ['the', 'man', 'is', 'wearing', 'a', 'black', 'jacket']
IDs:    [1996, 2158, 2003, 7078, 1037, 2304, 7505]
Especial: [CLS] = 101, [SEP] = 102
```

Vocabulario: 30 522 tokens base. Más 1 (índice 0 reservado para padding) $\approx$ 30 609 únicos. Más 1 para [PAD] = 29 610 (lo que usa este proyecto).

8.2 ¿Qué es un embedding?

Un *embedding* es una representación densa (vector de números reales) de un token. En este proyecto:

- Cada token del vocabulario $\rightarrow$ vector de 512 números reales.
- Total de parámetros: $29\,610 \times 512 = 15,16\text{M}$.

Inicialización: $\mathcal{N}(0,1)$ por defecto. Durante el entrenamiento, estos vectores se ajustan para que tokens con significado similar tengan vectores cercanos.

8.3 ¿Qué es un padding?

Los textos del batch tienen longitudes distintas. Para meterlos en un tensor uniforme se *padean* a una longitud fija (100 en este proyecto):

```
"a man in red"      -> [101, 1037, 2158, 1999, 2417, 102] (6 tokens)
"a tall person"     -> [101, 1037, 4206, 2718, 102] (5 tokens)
```

Ambos se convierten a longitud 100 añadiendo ceros ($[\text{PAD}] = 0$) al final. El modelo **ignora** el padding porque:

1. El índice 0 nunca se usa como token real (es $[\text{PAD}]$).
2. El embedding de índice 0 está fijo en $(0, 0, \dots, 0)$.
3. La GRU recibe `orig_token_length` para saber dónde termina cada secuencia.

8.4 ¿Qué es una GRU?

Una *Gated Recurrent Unit* [?] es una versión simplificada de LSTM. Procesa una secuencia token por token y mantiene un *estado oculto* h_t :

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (\text{update gate}) \quad (8.1)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (\text{reset gate}) \quad (8.2)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t \odot h_{t-1}, x_t]) \quad (8.3)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (8.4)$$

Bidireccional significa que se corren dos GRU en paralelo: una hacia adelante y otra hacia atrás. Las salidas se *promedian* para tener en cuenta contexto en ambas direcciones.

Ventajas sobre LSTM:

- Menos parámetros (2 compuertas vs 3).
- Más rápido en secuencias cortas.

8.5 ¿Qué es pack-padded-sequence?

8.5.1 El problema

Si procesamos un batch con secuencias de longitudes $[10, 30, 5, 20]$ todas padeadas a 100, la GRU hace trabajo inútil en los $[\text{PAD}]$. Además los $[\text{PAD}]$ afectan el estado oculto: como $h_{100} = f(h_{99}, 0)$, el estado final está "contaminado" por 70 pasos de padding.

8.5.2 La solución

pack_padded_sequence de PyTorch:

1. **Ordena** las secuencias por longitud descendente.
2. **Elimina** los pasos de padding antes de pasar a la RNN.
3. La RNN procesa sólo los tokens reales.
4. pad_packed_sequence restaura el orden original con padding.

Requisito: pasar las longitudes reales en CPU (no GPU) porque el backend de PyTorch así lo requiere.

8.6 Código: GRULanguageNetwork

```
class GRULanguageNetwork(nn.Module):
    """Embedding + Dropout + GRU bidireccional."""

    def __init__(self, config=None):
        super(GRULanguageNetwork, self).__init__()
        if config is None:
            raise ValueError("`config` can not be none")

        # Capa de embedding
        # vocab_size = 29609 (BERT) + 1 (padding) = 29610
        # embedding_dim = 512
        # padding_idx = 0: el vector para índice 0 se mantiene en 0
        self.embedding = nn.Embedding(
            config.vocab_size,          # 29610
            config.embedding_dim,       # 512
            padding_idx=0
        )

        # Dropout tras el embedding (30%)
        self.dropout = nn.Dropout(config.dropout_rate) # 0.30

        # GRU bidireccional
        # input_size: 512 (dim del embedding)
        # hidden_size: 1024 (feature_length)
        # num_layers: 1
        # bidirectional=True: procesa en ambas direcciones
        # bias=False: ahorra parámetros
        self.gru = nn.GRU(
            config.embedding_dim,        # 512
            config.feature_length,       # 1024
            num_layers=config.num_layers, # 1
            bidirectional=True,
            bias=False
        )

    def forward(self, text_ids=None, text_length=None):
        """text_ids: (B, 100), text_length: (B,).
        returns: (B, 1024, 100, 1)"""
        # 1. Embedding: (B, 100) -> (B, 100, 512)
        text_embedding = self.embedding(text_ids)
        # 2. Dropout (solo en training mode)
        text_embedding = self.dropout(text_embedding)
        # 3. Procesar con BiGRU (usando pack_padded_sequence)
        feature = self.do_RNN_computation(text_embedding, text_length, self.gru)
        return feature
```

8.7 Código: do_RNN_computation

```
def do_RNN_computation(self, text_embedding, text_length, gru):
    """Procesa la secuencia con GRU usando pack_padded_sequence."""

    # Asegurar que text_length es 1-D
    text_length = text_length.view(-1)

    # Ordenar índices por longitud DESCENDENTE
    _, sort_index = torch.sort(text_length, dim=0, descending=True)

    # Invertir el orden: nos servirá para "des-ordenar" después
    _, unsort_index = sort_index.sort()

    # Reordenar el batch por longitud descendente
    sortlength_text_embedding = text_embedding[sort_index, :]
    sort_text_length = text_length[sort_index]

    # Empaquetar: ignora el padding en el cómputo de la GRU
    packed_text_embedding = pack_padded_sequence(
        sortlength_text_embedding,
        sort_text_length.cpu(),
        batch_first=True
    )

    # Pasar por la GRU
    packed_feature, hn = gru(packed_text_embedding)

    # Desempaquetar: vuelve a la forma (B, T, *)
    total_length = text_embedding.size(1)
    sort_feature = pad_packed_sequence(
        packed_feature, batch_first=True, total_length=total_length
    )

    # Des-ordenar: volver al orden original del batch
    unsort_feature = sort_feature[0][unsort_index, :]

    # Promediar fwd y bwd:
    # BiGRU output: (B, T, 2*hidden) = (B, 100, 2048)
    #[:, :, :1024] = forward,[:, :, 1024:] = backward
    unsort_feature = (
        unsort_feature[:, :, :int(unsort_feature.size(2) / 2)] +
        unsort_feature[:, :, int(unsort_feature.size(2) / 2):]
    ) / 2

    # Preparar para alimentar el DSC final:
    # permute(0,2,1): (B, 100, 1024) -> (B, 1024, 100)
    # unsqueeze(3): añadir dim al final -> (B, 1024, 100, 1)
    return unsort_feature.permute(0, 2, 1).contiguous().unsqueeze(3)
```

8.8 Código: BERTTokenizer

```
class BERTTokenizer(object):
    """Adapta el tokenizer de HuggingFace a la interfaz del proyecto."""

    def __init__(self):
        super(BERTTokenizer, self).__init__()
```

```

# Importar transformers
_, tokenizer_class, pretrained_weights = (
    ppb.BertModel, ppb.BertTokenizer, 'bert-base-uncased'
)

# Cargar el tokenizer preentrenado (BERT-base, vocab inglés)
self.tokenizer = tokenizer_class.from_pretrained(pretrained_weights)

def __call__(self, text: str = None) -> torch.LongTensor:
    """Codifica texto a IDs de tokens."""
    # .encode() añade automáticamente [CLS] y [SEP]
    tokens = self.tokenizer.encode(text)
    return tokens

```

8.9 Código: pad_tokens

```

def pad_tokens(tokens, tokens_length_max):
    """Padea o trunca tokens a longitud fija."""
    tokens_length = len(tokens)

    # Convertir a tensor 1-D
    tokens = torch.from_numpy(np.array(tokens)).view(-1).contiguous().float()

    if tokens_length < tokens_length_max:
        # Caso corto: añadir zeros de padding al final
        zero_padding = torch.zeros(tokens_length_max - tokens_length)
        tokens = torch.cat([tokens, zero_padding], 0)
    else:
        # Caso largo: truncar a tokens_length_max
        tokens = tokens[:tokens_length_max]
        tokens_length = tokens_length_max

    return tokens, tokens_length

```

8.10 Flujo de tensores

```

1 # Entrada
2 text_ids:      (B, 100)      LongTensor, padded con 0s
3 text_length:  (B,)          longitudes reales (5-50 típico)
4
5 # 1. Embedding
6 out = embedding(text_ids)
7 # (B, 100) -> (B, 100, 512)
8
9 # 2. Dropout (solo training)
10 out = dropout(out)
11 # (B, 100, 512)
12
13 # 3. BiGRU con pack_padded_sequence
14 out = do_RNN_computation(out, text_length, self.gru)
15 # (B, 100, 512) -> (B, 100, 2048) -> (B, 100, 1024) -> (B, 1024, 100, 1)
16
17 # 4. Max-pool sobre tokens (en text_embedding)
18 out = max(out, dim=2)
19 # (B, 1024, 1, 1)
20
21 # 5. DSC compartido

```

```

22 out = depthwise_seperable_convolution(out)
23 # (B, 1024, 1, 1)
24
25 # 6. squeeze
26 out = out.squeeze(-1).contiguous()
27 # (B, 1024, 1)

```

8.11 Parámetros

Tabla 8.1: Parámetros de la red de lenguaje.

Componente	Parámetros
Embedding(29610, 512)	15,16 M
Dropout(0.30)	0
BiGRU(512→1024, bidir)	4,72 M
DSC(1024→1024)	10 K
Total lenguaje	19,89 M

Capítulo 9

TextReIDNet: joint embedding visual-textual

Este capítulo explica los conceptos detrás del modelo principal y muestra su código línea por línea.

9.1 ¿Qué es un joint embedding?

9.1.1 El concepto

Un *joint embedding space* (espacio de embedding conjunto) es un espacio vectorial donde las dos modalidades — imágenes y texto — se mapean al mismo lugar. La idea clave:

- Imagen de una persona $\rightarrow$ vector $\mathbf{I}' \in \mathbb{R}^{1024}$.
- Descripción de esa persona $\rightarrow$ vector $\mathbf{T}' \in \mathbb{R}^{1024}$.
- **Si el modelo es bueno, los vectores están *cerca* en el espacio.**

Entonces medir qué tan cerca están dos vectores nos dice qué tan parecidas son las dos entradas.

9.2 ¿Qué es la similitud coseno?

Dados dos vectores $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|} \in [-1, +1] \quad (9.1)$$

Interpretación:

- +1: vectores idénticos (apuntan en la misma dirección).
- 0: ortogonales (no relacionados).
- -1: opuestos.

Por qué coseno y no distancia euclidiana?:

- Invariante a la magnitud: $\mathbf{a}$ y $2\mathbf{a}$ tienen similitud 1.
- Es un *ángulo* entre vectores, no una distancia.
- Valores acotados en $[-1, +1]$, fácil de interpretar.

9.3 ¿Por qué un DSC compartido?

La última capa (`DepthwiseSeparableConv(1024, 1024)`) se usa para **ambas** ramas (visual y textual). Esto tiene dos efectos:

1. **Menos parámetros:** una capa en lugar de dos.

2. **Mismo espacio:** las dos modalidades se proyectan con la misma transformación, lo que fuerza a que vivan en el mismo espacio.

Analogía: es como si dos personas midieran en metros y otra persona midiera en pies. Si no comparten unidad, no se pueden comparar. El DSC compartido es la conversión a la misma unidad".

9.4 ¿Qué hace `squeeze` y `contiguous`?

9.4.1 `squeeze(dim)`

Elimina dimensiones de tamaño 1. Por ejemplo:

- $(B, 1024, 1, 1) \xrightarrow{\text{squeeze}(-1)} (B, 1024, 1)$.
- $(B, 1024, 1) \xrightarrow{\text{squeeze}(-1)} (B, 1024)$.

El argumento -1 indica "la última dimensión". Útil para preparar tensores para operaciones que esperan un número específico de dims.

9.4.2 `contiguous()`

Algunas operaciones de PyTorch (especialmente `view`, `reshape`, `slicing`) requieren que el tensor sea *contiguo* en memoria. Si no lo es, se hace una copia interna. Llamar `.contiguous()` fuerza la contigüidad explícitamente.

9.5 ¿Por qué dimensión 1024?

La dimensión del espacio conjunto (1024) es un compromiso:

- **Suficientemente grande** para capturar la semántica de ambas modalidades.
- **Suficientemente pequeño** para:
 - Similitud coseno estable numéricamente.
 - Inferencia rápida (4 KB por embedding en FP32).
 - Cabe en GPUs modestas.
- **Coincide** con las unidades ocultas de la BiGRU, lo que permite compartir el último DSC sin mismatch de dimensiones.

9.6 Código: Constructor `__init__`

```
class TextReIDNet(nn.Module):
    """La clase principal del modelo."""

    def __init__(self, configs: dict = None):
        # Llama al constructor de nn.Module (registra parámetros,
        # buffers, submódulos, etc.)
        super(TextReIDNet, self).__init__()

        # Validación: configs es obligatorio
        if configs is None:
            raise ValueError("`configs` parameter can not be None")

        # Guardar referencia a la configuración completa
        self.configs = configs

        # --- Rama visual ---
        # VisualNetwork = EfficientNet-B0 + wrapper
        self.visual_network = VisualNetwork()
```

```

# --- Rama de lenguaje ---
# GRULanguageNetwork = Embedding + Dropout + BiGRU
self.language_network = GRULanguageNetwork(configs)

# --- Capas de proyección al espacio conjunto ---
# 1ra DSC: reduce canales de 1280 (EffNet) a 1024 (joint dim)
self.visual_features_downscale = DepthwiseSeparableConv(1280, 1024)

# Adaptive max pooling: colapsa (H, W) a (1, 1)
self.adaptive_max_pooling = nn.AdaptiveMaxPool2d((1, 1))

# 2da DSC: mantiene 1024 canales, proyecta al espacio conjunto
# COMPARTIDA con la rama textual (ver text_embedding)
self.depthwise_seperable_convolution = DepthwiseSeparableConv(
    1024, self.configs.feature_length
)

```

9.7 Código: forward

```

def forward(self, image, text_ids=None, text_length=None):
    """image: (B,3,H,W), text_ids: (B,100), text_length: (B,)
    returns: ((B,1024,1), (B,1024,1))"""
    # Calcular embedding visual (rama izquierda)
    visauual_embeddings = self.image_embedding(image)

    # Calcular embedding textual (rama derecha)
    textual_embeddings = self.text_embedding(text_ids, text_length)

    # Devolver ambos. train.py los aplana a (B, 1024) con view(-1).
    # Nota: hay un typo en el código: "visauual" en lugar de "visual"
    return visauual_embeddings, textual_embeddings

```

9.8 Código: image_embedding

```

def image_embedding(self, image=None):
    """image: (B, 3, H, W) batch de imágenes.
    returns: (B, 1024, 1) embeddings visuales."""

    # PASO 1: EfficientNet-B0
    # Entrada: (B, 3, 512, 512)
    # Salida: (B, 1280, 16, 16) <- última etapa
    visual_features = self.visual_network(image)

    # PASO 2: DSC 1280 -> 1024
    # Entrada: (B, 1280, 16, 16)
    # Salida: (B, 1024, 16, 16)
    visual_features = self.visual_features_downscale(visual_features)

    # PASO 3: Adaptive Max Pooling
    # Entrada: (B, 1024, 16, 16)
    # Salida: (B, 1024, 1, 1)
    visual_features = self.adaptive_max_pooling(visual_features)

    # PASO 4: DSC final 1024 -> 1024 (compartido con texto)
    # Entrada: (B, 1024, 1, 1)

```

```

# Salida: (B, 1024, 1, 1)
visual_features = self.depthwise_seperable_convolution(visual_features)

# PASO 5: squeeze + contiguous
# .squeeze(-1) elimina la última dim (1) -> (B, 1024, 1)
# .contiguous() asegura memoria contigua
visual_features = visual_features.squeeze(-1).contiguous()
return visual_features

```

9.9 Código: text_embedding

```

def text_embedding(self, text_ids=None, text_length=None):
    """text_ids: (B, 100), text_length: (B,).
    returns: (B, 1024, 1) embeddings textuales."""

    # PASO 1: Pasar por la red de lenguaje
    # Salida de GRULanguageNetwork:
    # Entrada: (B, 100) tokens BERT
    # Embedding: (B, 100, 512)
    # BiGRU: (B, 100, 2048) bidireccional
    # avg f+b: (B, 100, 1024) promedio de fwd + bwd
    # permute: (B, 1024, 100, 1)
    textual_features = self.language_network(text_ids, text_length)

    # PASO 2: Max-pooling sobre la dim de tokens (dim=2)
    # Para cada canal (1024), tomar el valor máximo sobre los 100 tokens
    # Entrada: (B, 1024, 100, 1)
    # Salida: (B, 1024, 1, 1)
    textual_features, _ = torch.max(textual_features, dim=2, keepdim=True)

    # PASO 3: DSC compartido con la rama visual
    textual_features = self.depthwise_seperable_convolution(textual_features)

    # PASO 4: squeeze + contiguous
    textual_features = textual_features.squeeze(-1).contiguous()
    return textual_features

```

9.10 Diagrama del flujo completo

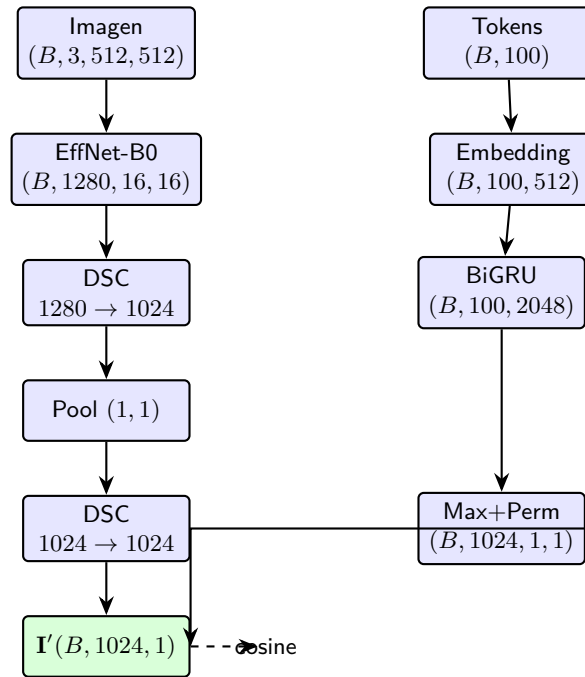


Figura 9.1: Flujo completo de tensores. Notar que el último DSC es compartido entre ambas ramas.

9.11 Inicialización de pesos

TextReIDNet no llama ninguna función de inicialización. Los pesos heredan:

- **EfficientNet-B0**: preentrenado en ImageNet.
- **Embedding**: $\mathcal{N}(0, 1)$ por defecto de PyTorch.
- **BiGRU**: inicialización por defecto de PyTorch.
- **DSC**: Kaiming (`weights_init_kaiming`).

9.12 Inferencia: extraer embeddings

```

1  import torch
2  from config import sys_configuration
3  from model.textreidnet import TextReIDNet
4
5  cfg = sys_configuration(dataset_source='huggingface')
6  model = TextReIDNet(cfg).to(cfg.device).eval()
7
8  # Modo evaluación (desactiva dropout)
9  with torch.no_grad():
10     # Embedding de imagen
11     img = torch.randn(2, 3, 512, 512).to(cfg.device)
12     img_emb = model.image_embedding(img) # (2, 1024, 1)
13     img_emb = img_emb.view(img_emb.size(0), -1) # (2, 1024)
14
15     # Embedding de texto
16     txt = torch.zeros(2, 100, dtype=torch.long).to(cfg.device)
17     lng = torch.tensor([50, 30]).to(cfg.device)
18     txt_emb = model.text_embedding(txt, lng) # (2, 1024, 1)

```

```
19     txt_emb = txt_emb.view(txt_emb.size(0), -1)  # (2, 1024)
20
21     # Similitud coseno entre las 2 imágenes y los 2 textos
22     img_emb_n = img_emb / img_emb.norm(dim=1, keepdim=True)
23     txt_emb_n = txt_emb / txt_emb.norm(dim=1, keepdim=True)
24     sim = torch.mm(img_emb_n, txt_emb_n.t())      # (2, 2)
25     print(sim)
```

Capítulo 10

Funciones de pérdida: triplet y cross-entropy

Este capítulo explica los conceptos detrás de las dos pérdidas usadas y muestra su código línea por línea.

10.1 ¿Qué es un embedding y por qué necesita pérdidas?

Después del forward tenemos dos vectores por muestra:

- $\mathbf{I}' \in \mathbb{R}^{1024}$ (imagen).
- $\mathbf{T}' \in \mathbb{R}^{1024}$ (texto).

Objetivo: vectores de la misma persona deben estar *cerca*, y de personas diferentes *lejos*.

Problema: el modelo no sabe cómo hacerlo solo. Necesitamos una función que mida qué tan mal lo está haciendo y guíe el aprendizaje: la **pérdida** (loss).

10.2 ¿Qué es la triplet loss?

10.2.1 Concepto

La *triplet loss* [?] trabaja con *triplets*: tríos de muestras (anchor, positive, negative).

- **Anchor** (A): muestra de referencia.
- **Positive** (P): muestra del *mismo* pid.
- **Negative** (N): muestra de *otro* pid.

La pérdida enforces que:

$$\text{sim}(A, P) - \text{sim}(A, N) + \text{margin} \leq 0 \quad (10.1)$$

En palabras: el anchor debe estar *más cerca* del positive que del negative, con un margen mínimo (margin = 0,5 en este proyecto).

10.2.2 ¿Por qué el margen?

Sin margen, basta con que $\text{sim}(A, P) > \text{sim}(A, N)$. Esto da una solución trivial: todos los embeddings en un punto. Con margen exigimos que la diferencia sea $\geq \text{margin}$, lo que *obliga* a aprender embeddings informativos.

10.3 ¿Qué es semi-hard mining?

10.3.1 Los tres tipos de negativos

Para un anchor dado, hay tres tipos de negativos:

1. **Easy**: $\text{sim}(A, N) \ll \text{sim}(A, P)$. El modelo ya los clasifica bien. *No ayudan* al aprendizaje.
2. **Hard**: $\text{sim}(A, N) > \text{sim}(A, P)$. El modelo ya los confunde. Contribuyen mucho al gradiente pero pueden hacer el entrenamiento inestable.
3. **Semi-hard**: $\text{sim}(A, P) < \text{sim}(A, N) < \text{sim}(A, P) + \text{margin}$. El modelo está cerca pero no del todo mal. *Ideal*: gradiente útil + entrenamiento estable.

10.3.2 Criterio usado en este proyecto

El código implementa semi-hard mining [?]:

- Si hay negativos semi-hard disponibles, elegir uno aleatoriamente.
- Si no hay ninguno, usar uno cualquiera (caso degenerado).

Ventaja: estable y eficiente; converge más rápido que easy mining sin los problemas de hard mining.

10.4 ¿Qué es la cross-entropy?

10.4.1 Clasificación como identidad

Otra forma de entrenar el modelo es agregar una cabeza de **clasificación** que predice el pid (identidad de persona). Si el modelo predice correctamente, está discriminando bien.

La cross-entropy mide qué tan lejos están las predicciones del label correcto:

$$\mathcal{L}_{\text{CE}} = - \sum_{c=1}^C y_c \cdot \log(\hat{y}_c) \quad (10.2)$$

donde y_c es 1 si c es el pid correcto, $\hat{y}_c$ es la probabilidad predicha para la clase c .

10.4.2 PyTorch lo hace solo

`nn.CrossEntropyLoss(reduction='mean')` computa la CE promediada sobre el batch. Internamente aplica `LogSoftmax + NLLLoss`.

10.5 ¿Por qué simétrica (i2t + t2i)?

Tanto `RankingLoss` como `IdentityLoss` se calculan en **ambas direcciones**:

- **i2t**: predice la clase del texto desde el embedding de la imagen.
- **t2i**: predice la clase de la imagen desde el embedding del texto.

¿Por qué? El modelo debe ser bueno *simétricamente*: si una imagen $\leftrightarrow$ texto describe a la misma persona, ambos sentidos deben predecir el mismo pid. Esto fuerza coherencia en ambos lados del embedding.

10.6 Pérdida total

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{rank}} + \beta \cdot \mathcal{L}_{\text{ID}} \quad (10.3)$$

En este proyecto $\alpha = \beta = 1,0$ (config.py:87-88).

10.7 Código: calculate_similarity

```
def calculate_similarity(image_embedding, text_embedding):
    """Calcula la matriz de similitud coseno entre imágenes y textos."""

    # Aplanar embeddings a (B, 1024) por si vienen con más dims
    image_embedding = image_embedding.view(image_embedding.size(0), -1)
    text_embedding = text_embedding.view(text_embedding.size(0), -1)

    # Normalizar L2 (norma = 1)
    # +1e-8 para evitar división por cero
    image_embedding_norm = image_embedding / (
        image_embedding.norm(dim=1, keepdim=True) + 1e-8
    )
    text_embedding_norm = text_embedding / (
        text_embedding.norm(dim=1, keepdim=True) + 1e-8
    )

    # Producto matricial: similitud entre cada par (img_i, txt_j)
    similarity = torch.mm(image_embedding_norm, text_embedding_norm.t())
    return similarity
```

10.8 Código: RankingLoss

```
class RankingLoss(nn.Module):
    """Triplet loss con minería semi-hard."""

    def __init__(self, config=None):
        super(RankingLoss, self).__init__()
        self.config = config

    def semi_hard_negative(self, loss):
        """Elige un negativo semi-hard al azar."""
        # loss: array con losses para cada negativo
        # logical_and(0 < loss < margin)
        negative_index = np.where(
            np.logical_and(loss < self.config.margin, loss > 0)
        )[0]
        return np.random.choice(negative_index) if len(negative_index) > 0 else None

    def get_triplets(self, similarity, labels):
        """Para cada anchor, busca un par (positivo, negativo)."""
        similarity = similarity.cpu().data.numpy()
        labels = labels.cpu().data.numpy()
        triplets = []

        for idx, label in enumerate(labels):
            # Índices con OTRO pid (negativos)
            negative = np.where(labels != label)[0]
            # Similitud del anchor consigo mismo (~1)
            ap_sim = similarity[idx, idx]
            # loss = sim(a,n) - sim(a,p) + margin
            loss = similarity[idx, negative] - ap_sim + self.config.margin
            # Elegir un negativo semi-hard
            negative_index = self.semi_hard_negative(loss)
            if negative_index is not None:
                triplets.append([idx, idx, negative[negative_index]])
```

```

if len(triplets) == 0:
    triplets.append([idx, idx, negative[0]])
triplets = np.array(triplets)
return torch.LongTensor(triplets)

```

```

def forward(self, img, txt, label, sim_neg=None):
    """img: (B,1024), txt: (B,1024), label: (B,)"""
    similarity = calculate_similarity(img, txt)
    if sim_neg is not None:
        similarity = similarity + sim_neg

    # Triplets image->text
    image_triplets = self.get_triplets(similarity, label)
    image_anchor_loss = F.relu(
        self.config.margin
        - similarity[image_triplets[:, 0], image_triplets[:, 1]]
        + similarity[image_triplets[:, 0], image_triplets[:, 2]]
    )

    # Triplets text->image (transponer)
    text_triplets = self.get_triplets(similarity.t(), label)
    similarity = similarity.t()
    texty_anchor_loss = F.relu(
        self.config.margin
        - similarity[text_triplets[:, 0], text_triplets[:, 1]]
        + similarity[text_triplets[:, 0], text_triplets[:, 2]]
    )

    loss = torch.sum(image_anchor_loss) + torch.sum(texty_anchor_loss)
    return loss

```

10.9 Código: IdentityLoss

```

def weights_init_classifier(m):
    """Inicializa capas Linear con N(0, 0.001) y bias=0."""
    classname = m.__class__.__name__
    if classname.find('Linear') != -1:
        init.normal_(m.weight.data, std=0.001)
        init.constant_(m.bias.data, 0.0)

class classifier(nn.Module):
    """Capa lineal simple: input_dim -> output_dim."""

    def __init__(self, input_dim, output_dim):
        super(classifier, self).__init__()
        self.block = nn.Linear(input_dim, output_dim)
        self.block.apply(weights_init_classifier)

    def forward(self, x):
        return self.block(x)

```

```

class IdentityLoss(nn.Module):
    """Cross-entropy simétrica: image->text + text->image."""

    def __init__(self, config=None, part=1, class_num=None):

```

```

super(IdentityLoss, self).__init__()
self.part = part

# Crear `part` clasificadores (en este proyecto part=1)
W = []
for i in range(part):
    W.append(classifier(config.feature_length, class_num))
self.W = nn.Sequential(*W)

def calculate_IdLoss(self, image_embedding_local,
                    text_embedding_local, label):
    """image_embedding_local: (B,1024,1), text_embedding_local: (B,1024,1)"""
    label = label.view(label.size(0))
    criterion = nn.CrossEntropyLoss(reduction='mean')

    Lipt_local = 0 # image->text loss
    Ltpi_local = 0 # text->image loss

    for i in range(self.part):
        # Proyectar embedding al espacio de clases
        score_i2t_local_i = self.W[i](image_embedding_local[:, :, i])
        score_t2i_local_i = self.W[i](text_embedding_local[:, :, i])

        Lipt_local += criterion(score_i2t_local_i, label)
        Ltpi_local += criterion(score_t2i_local_i, label)

    loss = (Lipt_local + Ltpi_local) / self.part
    return loss

def forward(self, image_embedding_local,
            text_embedding_local, label):
    return self.calculate_IdLoss(image_embedding_local,
                                text_embedding_local, label)

```

10.10 Uso conjunto en train.py

```

with torch.autocast(device_type=config.device, dtype=torch.float16):
    # Forward
    visual_embeddings, textual_embeddings = model(
        image=preprocessed_images,
        text_ids=token_ids,
        text_length=orig_token_length
    )

    # Aplanar a (B, 1024) - RankingLoss espera aplanados
    visual_emb_flat = visual_embeddings.view(visual_embeddings.size(0), -1)
    text_emb_flat = textual_embeddings.view(textual_embeddings.size(0), -1)
    labels_flat = labels.view(-1)

    # RankingLoss: triplet semi-hard
    ranking_loss = ranking_loss_fnx(visual_emb_flat, text_emb_flat, labels_flat)

    # IdentityLoss: cross-entropy simétrica
    identity_loss = identity_loss_fnx(visual_embeddings, textual_embeddings, labels_flat)

    # Pérdida total: combinación ponderada
    total_loss = (config.ranking_loss_alpha * ranking_loss
                  + config.identity_loss_beta * identity_loss)

```

Capítulo 11

Entrenamiento: optimizador, scheduler y mixed precision

Este capítulo explica los conceptos detrás del loop de entrenamiento y muestra el código línea por línea.

11.1 ¿Qué es AdamW?

11.1.1 Adam básico

Adam (*Adaptive Moment Estimation*) [?] es un optimizador que mantiene dos estadísticas por parámetro:

- m_t : promedio exponencial del gradiente (momento 1).
- v_t : promedio exponencial del gradiente al cuadrado (momento 2).

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (11.1)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (11.2)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (11.3)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (11.4)$$

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (11.5)$$

Ventaja: adapta el learning rate por parámetro, lo que acelera la convergencia en problemas con gradientes de magnitudes muy distintas.

11.1.2 AdamW

AdamW [?] separa el *weight decay* del optimizador, en vez de aplicarlo como una regularización L2 en la loss. Esto da una regularización más efectiva.

En este proyecto:

- $\beta_1 = 0,90$ (`config.adam_alpha`).
- $\beta_2 = 0,999$ (`config.adam_beta`).
- $lr = 0.001$ (`config.lr`).

11.2 ¿Qué es un LR scheduler?

Un *learning rate scheduler* modifica el learning rate durante el entrenamiento. Hay varios tipos:

- **Step**: decae en epochs fijos.
- **Exponential**: decae exponencialmente.
- **Cosine**: cae siguiendo un coseno.
- **ReduceOnPlateau**: decae si la loss no mejora.

Este proyecto usa **MultiStepLR**: decae por un factor de 0.1 en los epochs especificados en `epoch_decay = [20, 40]`.

- Epochs 1–19: $lr = 0.001$.
- Epochs 20–39: $lr = 0.0001$.
- Epochs 40–60: $lr = 0.00001$.

¿Por qué decaer? Al principio se quiere aprender rápido. Cerca del óptimo, lr grande hace oscilar; reducirlo permite converger.

11.3 ¿Qué es mixed precision (FP16/BF16)?

11.3.1 Precisiones disponibles

- **FP32** (32 bits): precisión estándar. Ocupa 4 bytes por número.
- **FP16** (16 bits): mitad de memoria. Acelera en GPUs con Tensor Cores (NVIDIA desde Volta).
- **BF16** (16 bits): mismo rango que FP32 pero menos precisión. *No necesita scaler*.

11.3.2 El problema del FP16

FP16 tiene rango muy limitado (± 65504). Si un gradiente es muy pequeño ($< 6 \cdot 10^{-5}$), se redondea a 0 y *desaparece*.

11.3.3 Solución: GradScaler

`torch.cuda.amp.GradScaler` multiplica la loss por un factor (típicamente 2^{16}) antes del `backward`, lo que efectivamente *amplifica* los gradientes. Luego los des-escala al hacer `step`:

```

1 scaler = GradScaler()
2 scaler.scale(loss).backward()      # loss * factor -> gradientes grandes
3 scaler.step(optimizer)              # des-escala y aplica
4 scaler.update()                    # ajusta el factor dinámicamente

```

11.3.4 Autocast

`torch.autocast` aplica casting automático de tipos:

- Operaciones *seguras* (matmul, conv): en FP16.
- Operaciones *sensibles* (log, softmax, loss): en FP32.

Esto se aplica con un context manager:

```

1 with torch.autocast(device_type='cuda', dtype=torch.float16):
2     output = model(input)
3     loss = criterion(output, target)

```

11.3.5 GPU vs CPU

En este proyecto:

```

1 # GPU: FP16 con GradScaler
2 # CPU: BF16 sin scaler (más estable)
3 precision_dtype = torch.bfloat16 if config.device == 'cpu' \
4                     else torch.float16

```

11.4 ¿Qué hace `optimizer.zero_grad()`?

Por defecto PyTorch *acumula* gradientes entre llamadas a `backward()`. Hay que **limpiar** explícitamente los gradientes al inicio de cada batch:

```

1 optimizer.zero_grad() # <- SIEMPRE antes de forward
2 output = model(input)
3 loss = criterion(output, target)
4 loss.backward()      # acumula en .grad
5 optimizer.step()     # aplica los gradientes

```

Alternativa: `optimizer.zero_grad(set_to_None=True)` libera memoria al poner los gradientes en `None` en vez de 0.

11.5 ¿Cómo se guarda un checkpoint?

Un checkpoint es un diccionario con todo lo necesario para reanudar el entrenamiento:

```

1 checkpoint = {
2     'epoch': current_epoch,
3     'model_state_dict': model.state_dict(),
4     'optimizer_state_dict': optimizer.state_dict(),
5     'loss': current_loss
6 }
7 torch.save(checkpoint, 'checkpoint.pth.tar')

```

state_dict: diccionario con todos los parámetros del modelo (o del optimizer). Es lo que se mueve a GPU/CPU con `model.load_state_dict()`.

11.6 Código: `train.py` main + loop

```

def parse_args():
    parser = argparse.ArgumentParser(description='Train TextReIDNet on CUHK-PEDES')

    # Fuente del dataset: 'huggingface' (rápido) o 'original' (paper)
    parser.add_argument('--dataset_source', type=str, default='huggingface',
                        choices=['huggingface', 'original'])
    parser.add_argument('--epochs', type=int, default=None)
    parser.add_argument('--batch_size', type=int, default=None)
    parser.add_argument('--lr', type=float, default=None)
    parser.add_argument('--seed', type=int, default=None)
    parser.add_argument('--output_dir', type=str, default='./data/checkpoints')

    return parser.parse_args()

```

```

def main():
    args = parse_args()

    # Construir config desde config.py
    config = sys_configuration(
        dataset_name="CUHK-PEDES",
        dataset_source=args.dataset_source
    )

    # Aplicar overrides de CLI sobre la config
    if args.epochs is not None:
        config['epoch'] = args.epochs
    if args.batch_size is not None:
        config['batch_size'] = args.batch_size
    if args.lr is not None:
        config['lr'] = args.lr
    if args.seed is not None:
        config['seed'] = args.seed
    if args.output_dir is not None:
        config['model_save_path'] = os.path.abspath(args.output_dir)
        os.makedirs(config['model_save_path'], exist_ok=True)

    # Fijar semilla para reproducibilidad
    set_seed(config.seed)

    # Compartir memoria via filesystem (evita errores con muchos workers)
    torch.multiprocessing.set_sharing_strategy('file_system')

    # GradScaler para mixed precision (FP16/BF16)
    scaler = GradScaler()

    # Construir DataLoaders
    train_data_loader, _, _, train_num_classes = \
        build_cuhkpedes_dataloader(config)

    # Construir modelo y mover a device
    model = TextReIDNet(config).to(config.device)

    # IdentityLoss tiene parámetros entrenables (la capa Linear)
    identity_loss_fnx = IdentityLoss(
        config=config, class_num=train_num_classes
    ).to(config.device)

    # RankingLoss NO tiene parámetros entrenables
    ranking_loss_fnx = RankingLoss(config)

    # Optimizer: AdamW
    optimizer = optim.AdamW(
        model.parameters(),
        betas=(config.adam_alpha, config.adam_beta),
        lr=config.lr
    )

    # Scheduler: decae lr en epochs específicos
    scheduler = optim.lr_scheduler.MultiStepLR(optimizer, config.epoch_decay)

```

```

for current_epoch in range(1, config.epoch + 1):
    train_ranking_loss_list = []
    train_identity_loss_list = []

```

```

train_total_loss_list = []

model.train()

with tqdm(train_data_loader, unit='batch') as tepoch:
    tepoch.set_description(f"Epoch {current_epoch}/{config.epoch}")

    for batch in tepoch:
        # Mover datos al device
        preprocessed_images = batch['preprocessed_images'].to(config.device)
        labels = batch['pids'].to(config.device)
        token_ids = batch['token_ids'].to(config.device)
        orig_token_length = batch['orig_token_lengths'].to(config.device)

        # Limpiar gradientes del batch anterior
        optimizer.zero_grad()

        # Mixed precision
        precision_dtype = torch.bfloat16 if config.device == 'cpu' \
            else torch.float16
        with torch.autocast(device_type=config.device, dtype=precision_dtype):
            # FORWARD
            visual_embeddings, textual_embeddings = model(
                image=preprocessed_images,
                text_ids=token_ids,
                text_length=orig_token_length
            )

            # Aplanar para losses
            visual_emb_flat = visual_embeddings.view(visual_embeddings.size(0), -1)
            text_emb_flat = textual_embeddings.view(textual_embeddings.size(0), -1)
            labels_flat = labels.view(-1)

            # Calcular losses
            ranking_loss = ranking_loss_fnx(visual_emb_flat, text_emb_flat, labels_flat)
            identity_loss = identity_loss_fnx(visual_embeddings, textual_embeddings, labels_flat)
            total_loss = (config.ranking_loss_alpha * ranking_loss
                          + config.identity_loss_beta * identity_loss)

        # BACKWARD con scaler (para FP16)
        scaler.scale(total_loss).backward()

        # STEP: actualizar parámetros
        scaler.step(optimizer)
        scaler.update()

        # Acumular losses para reporting
        train_ranking_loss_list.append(ranking_loss.item())
        train_identity_loss_list.append(identity_loss.item())
        train_total_loss_list.append(total_loss.item())

    tepoch.set_postfix({
        'rank': f"{np.mean(train_ranking_loss_list):.3f}",
        'id': f"{np.mean(train_identity_loss_list):.3f}",
        'tot': f"{np.mean(train_total_loss_list):.3f}",
    })

scheduler.step()

# Guardar checkpoint
ckpt_path = os.path.join(
    config.model_save_path,

```

```

        f"TextReIDNet_epoch{current_epoch}.pth.tar"
    )
    torch.save({
        'epoch': current_epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'loss': np.mean(train_total_loss_list),
    }, ckpt_path)

    latest_path = os.path.join(
        config.model_save_path, "TextReIDNet_latest.pth.tar"
    )
    torch.save({
        'epoch': current_epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'loss': np.mean(train_total_loss_list),
    }, latest_path)

```

11.7 Estimación de tiempo

Con los parámetros default (60 epochs, batch 8, FP16):

- **RTX 2060**: 2–4 días.
- **RTX 3090**: 16–24 horas.
- **A100**: 8–12 horas.

Nota: el HF dataset tiene 179k muestras (vs 40k del original). El entrenamiento es proporcionalmente más largo que el paper.

11.8 Caveats

1. **Sin early stopping:** el código entrena los 60 epochs completos. `patience = 3` está en config pero no se usa.
2. **Sin validación intermedia:** no evalúa sobre val/test durante el entrenamiento.
3. **Sin accumulation de gradientes:** si OOM, hay que bajar `batch_size`.
4. **Checkpoint por epoch:** 61 archivos `.pth.tar` (~250 MB cada uno para 25M parámetros).

Capítulo 12

Evaluación: extracción de features y métricas

Este capítulo explica los conceptos de inferencia y muestra el código de evaluación. Para la explicación detallada de las métricas (Top-k, mAP, AP), ver el Capítulo 13.

12.1 ¿Qué es la inferencia (evaluación)?

Una vez entrenado el modelo, queremos medir su rendimiento. El proceso tiene tres pasos:

1. **Extraer embeddings** de todas las imágenes y textos del split test.
2. **Computar la matriz de similitud coseno** entre imágenes y textos.
3. **Calcular métricas** (Top-k, mAP) usando los pids.

Diferencia con entrenamiento:

- En entrenamiento se hace forward + backward.
- En inferencia sólo forward (sin gradientes).

12.2 ¿Por qué `@torch.no_grad()`?

`@torch.no_grad()` es un decorator que desactiva el cálculo de gradientes. Beneficios:

- **Menor memoria:** no se almacenan gradientes.
- **Más rápido:** ahorra cómputo.
- **Previene errores:** garantiza que no se modifiquen parámetros accidentalmente.

12.3 ¿Qué es `model.eval()`?

`model.eval()` cambia el modo del modelo:

- Desactiva **Dropout** (no se eliminan activaciones).
- Usa estadísticas *acumuladas* en **BatchNorm** en lugar de las del batch.
- Afecta a capas como **Dropout**, **BatchNorm**.

Para inferir SIEMPRE se debe llamar `model.eval()` antes.

12.4 Galería vs query: convención de este proyecto

En este proyecto:

- **Query** = texto (descripción de la persona).
- **Gallery** = imágenes.

Para cada query se buscan imágenes similares en la gallery. La métrica **Top-k** indica cuántas queries tienen la imagen correcta en el top-k.

12.5 Código: extracción de features de imágenes

```
@torch.no_grad()
def extract_image_features(model, img_loader, device):
    """Extrae embeddings visuales para todas las imágenes del loader."""

    # Modo evaluación (desactiva dropout, etc.)
    model.eval()

    all_features = []
    all_pids = []
    all_paths = []

    for batch in tqdm(img_loader, desc='Extracting image features'):
        if isinstance(batch, (list, tuple)):
            imgs, pids, paths = batch[0], batch[1], batch[2]
        else:
            imgs, pids, paths = batch

        # GPU
        imgs = imgs.to(device)

        # Forward solo en la rama visual
        # image_embedding retorna (B, 1024, 1)
        feats = model.image_embedding(imgs)

        # Aplanar y mover a CPU
        feats = feats.view(feats.size(0), -1).cpu()

        all_features.append(feats)
        all_pids.extend(pids.tolist() if torch.is_tensor(pids) else list(pids))
        all_paths.extend(paths if isinstance(paths, list) else [paths])

    # Concatenar todos los batches
    return torch.cat(all_features, 0), all_pids, all_paths
```

12.6 Código: extracción de features de textos

```
@torch.no_grad()
def extract_text_features(model, txt_loader, device):
    """Extrae embeddings textuales."""

    model.eval()

    all_features = []
    all_pids = []

    for batch in tqdm(txt_loader, desc='Extracting text features'):
        if isinstance(batch, (list, tuple)):
            pids, token_ids, token_lengths = batch[0], batch[1], batch[2]
        else:
            pids, token_ids, token_lengths = batch

        token_ids = token_ids.to(device)
        token_lengths = token_lengths.to(device)

        # Forward solo en la rama textual
        feats = model.text_embedding(token_ids, token_lengths)
```

```

    feats = feats.view(feats.size(0), -1).cpu()

    all_features.append(feats)
    all_pids.extend(pids.tolist() if torch.is_tensor(pids) else list(pids))

    return torch.cat(all_features, 0), all_pids

```

12.7 Código: similitud coseno

```

def calculate_similarity(image_feature_local, text_feature_local):
    """image: (N_img, 1024), text: (N_txt, 1024)
        returns: (N_img, N_txt) numpy array de similitudes coseno"""

    # Aplanar por si tienen más dims
    image_feature_local = image_feature_local.view(image_feature_local.size(0), -1)
    text_feature_local = text_feature_local.view(text_feature_local.size(0), -1)

    # Normalizar L2 (cada vector a norma 1)
    image_feature_local = image_feature_local / \
        image_feature_local.norm(dim=1, keepdim=True)
    text_feature_local = text_feature_local / \
        text_feature_local.norm(dim=1, keepdim=True)

    # Producto matricial: S[i,j] = cos(img_i, txt_j)
    similarity = torch.mm(image_feature_local, text_feature_local.t())

    # Devolver como numpy (requerido por calculate_ap)
    return similarity.cpu().numpy()

```

12.8 Código: pipeline completo de evaluación

```

def main():
    args = parse_args()

    config = sys_configuration(
        dataset_name="CUHK-PEDES",
        dataset_source=args.dataset_source
    )
    config['batch_size'] = args.batch_size
    config['device'] = args.device
    config['model_testing_data_split'] = args.split # 'test' o 'val'

    # Construir dataloaders (devuelve img_loader y txt_loader)
    _, img_loader, txt_loader, _ = build_cuhkpedes_dataloader(config)

    # Construir modelo y cargar checkpoint
    model = TextReIDNet(config).to(config.device)
    ckpt = torch.load(args.checkpoint, map_location=config.device)
    if 'model_state_dict' in ckpt:
        model.load_state_dict(ckpt['model_state_dict'])
    else:
        model.load_state_dict(ckpt) # compatibilidad

    # PASO 1: extraer features de imágenes
    img_feats, img_pids, img_paths = extract_image_features(
        model, img_loader, config.device

```

```

)

# PASO 2: extraer features de textos
txt_feats, txt_pids = extract_text_features(
    model, txt_loader, config.device
)

# PASO 3: matriz de similitud coseno
similarity = calculate_similarity(img_feats, txt_feats)

# PASO 4: calcular CMC y mAP
img_pids_t = torch.tensor(img_pids)
txt_pids_t = torch.tensor(txt_pids)
cmc, mAP = evaluate(similarity, txt_pids_t, img_pids_t)

# PASO 5: imprimir resultados
top1 = cmc[0] * 100
top5 = cmc[4] * 100 if len(cmc) > 4 else 0
top10 = cmc[9] * 100 if len(cmc) > 9 else 0

print(f" Top-1: {top1:.2f}%")
print(f" Top-5: {top5:.2f}%")
print(f" Top-10: {top10:.2f}%")
print(f" mAP: {mAP*100:.2f}%")

# Guardar en log
logger = setup_logger(name='eval_logger',
                      log_file_path=config.test_log_path,
                      write_mode='append')
logger.info(f"Top-1: {top1:.2f}% | Top-5: {top5:.2f}% | "
           f"Top-10: {top10:.2f}% | mAP: {mAP*100:.2f}%")

```

12.9 Uso desde la terminal

```

1 # Evaluar en el split test
2 .venv/bin/python evaluate.py \
3     --checkpoint data/checkpoints/TextReIDNet_latest.pth.tar \
4     --dataset_source huggingface \
5     --split test
6
7 # Cambiar batch size
8 .venv/bin/python evaluate.py \
9     --checkpoint data/checkpoints/TextReIDNet_latest.pth.tar \
10    --batch_size 64
11
12 # Forzar CPU (lento)
13 .venv/bin/python evaluate.py \
14    --checkpoint data/checkpoints/TextReIDNet_latest.pth.tar \
15    --device cpu

```

12.10 Resultados típicos

Tabla 12.1: Resultados esperados según el README.

Setup	Top-1	Top-5	Top-10	mAP
Sin entrenar (random)	~0.02 %	0.05 %	0.10 %	0.10 %
HF (128×128) 50 batches	0.02 %	0.08 %	0.17 %	0.10 %
HF entrenado completo	45–52 %	~70 %	~80 %	~40 %
Original paper	54.02 %	75.51 %	85.28 %	~50 %

12.11 Limitaciones

1. **Sin gallery augmentation:** cada imagen se evalúa una sola vez.
2. **Sin query expansion:** no se re-rankea usando los top-k.
3. **Sin análisis por longitud de caption.**
4. **Sin visualización** de aciertos/fallos.

Capítulo 13

Métricas: Top-k, AP, mAP y CMC

Este capítulo desglosa línea por línea cada una de las métricas que usa el proyecto. Si nunca trabajaste con *re-ID* o recuperación de imágenes, esta es la lectura obligatoria: vas a entender qué significa cada número del Top-k y cómo se calcula exactamente.

13.1 El problema de evaluación

El modelo produce, para cada imagen y cada texto, un vector de 1024 dimensiones (un *embedding*). Para evaluar, comparamos esos vectores con la **similitud coseno**:

$$\text{sim}(\mathbf{I}', \mathbf{T}') = \frac{\mathbf{I}' \cdot \mathbf{T}'}{\|\mathbf{I}'\| \cdot \|\mathbf{T}'\|} \quad (13.1)$$

El resultado es una **matriz de similitud** $\mathbf{S} \in \mathbb{R}^{N_{\text{img}} \times N_{\text{txt}}}$ donde S_{ij} indica qué tan parecida es la imagen i al texto j .

Galería vs. query

En este proyecto:

- **Query** = descripción textual (texto). Son $N_{\text{txt}} = 29\,930$ en el split test.
- **Gallery** = imágenes. Son $N_{\text{img}} = 4\,256$ en el split test.

Para cada query (texto) queremos encontrar la imagen correcta en la gallery. El ranking se obtiene ordenando las imágenes por similitud descendente.

13.2 Top-k accuracy (CMC)

Qué es

El *Top-k accuracy* (también llamado **CMC**, Cumulative Matching Characteristic) responde a la pregunta:

Dada una descripción textual, ¿la imagen correcta está entre las primeras k del ranking?

Ejemplo: si para el texto “*mujer con camiseta negra y pantalones caqui*” el ranking de imágenes es:

1. img_382 (correcta)
2. img_124
3. img_991

entonces Top-1 = 100 % (acertó de entrada), Top-5 = 100 % (también está en el top-5). Si la imagen correcta hubiera aparecido en la posición 7, Top-1 = 0 % pero Top-10 = 100 %.

Cómo se calcula

Para todos los queries a la vez:

$$\text{Top-}k = \frac{1}{N_q} \sum_{i=1}^{N_q} \mathbb{K}[\text{correcta}_i \leq k] \quad (13.2)$$

donde $\mathbb{K}[\cdot]$ es la función indicadora (1 si se cumple, 0 si no).

13.3 Average Precision (AP)

Qué es

El AP mide la calidad del ranking *completo*, no sólo del top-1. Es el área bajo la curva *precisión-recall*.

Ejemplo paso a paso

Supongamos que para una query hay **3 imágenes correctas** en la gallery y el modelo las rankea así (de mayor a menor similitud):

Tabla 13.1: Ejemplo de ranking para una query con 3 positivos.

Pos.	¿Correcta?	Precisión	Recall	Nota
1		1/1 = 1.00	1/3 = 0.33	primer positivo
2		0/2 = 0.00	1/3 = 0.33	falso positivo
3		1/3 = 0.33	2/3 = 0.67	segundo positivo
4		1/4 = 0.25	2/3 = 0.67	falso positivo
5		2/5 = 0.40	3/3 = 1.00	tercer positivo

Precisión = (correctos vistos) / (total vistos).

Recall = (correctos vistos) / (total correctos).

El AP es la integral de la curva precisión-recall. Numéricamente, la regla del trapecio da:

$$\text{AP} = \sum_{i=1}^N \Delta \text{recall}_i \cdot \frac{\text{prec}_i + \text{prec}_{i-1}}{2} \quad (13.3)$$

Aplicado a las posiciones donde hay positivos (1, 3, 5):

$$\begin{aligned}
 \Delta \text{recall}_1 &= 1/3 & \text{prec}_1 &= 1, \text{prec}_0 = 1 \\
 \Delta \text{recall}_3 &= 1/3 & \text{prec}_3 &= 1/3, \text{prec}_2 = 0 \\
 \Delta \text{recall}_5 &= 1/3 & \text{prec}_5 &= 2/5, \text{prec}_4 = 1/4 \\
 \text{AP} &= \frac{1}{3} \left(1 + \frac{0.5 + 0}{2} \right) + \frac{1}{3} \left(\frac{1}{3} + \frac{1}{4} \right) \cdot \frac{1}{2} \cdot 2 \\
 &= 0.667 + 0.097 + 0.097 \\
 &= 0.86
 \end{aligned}$$

Implementación línea por línea

```
def calculate_ap(similarity, label_query, label_gallery):
    """
    Args.:
        similarity:    vector 1-D con similitud a cada imagen de la gallery
        label_query:    pid (int) del query
        label_gallery:  vector 1-D con pids de la gallery (mismo orden que similarity)
    Returns: (ap, cmc) o (None, None) si no hay positivos
    """

    # Ordenar índices por similitud DESCENDENTE (mejor primero)
    # np.argsort devuelve ascending; [::-1] invierte el orden
    # np es numpy: import numpy as np
    index = np.argsort(similarity)[::-1]

    # good_index: posiciones en la gallery donde el pid == label_query
    # np.argwhere devuelve índices donde se cumple la condición
    good_index = np.argwhere(label_gallery == label_query)

    # cmc: vector de ceros del mismo tamaño que el ranking
    # Lo llenaremos con 1 desde la posición del primer match
    cmc = np.zeros(index.shape)

    # Máscara booleana: True donde index[i] está en good_index
    # np.in1d chequea pertenencia a un conjunto, eficiente
    mask = np.in1d(index, good_index)

    # Posiciones (en el ranking) donde hay match
    # precision_result[i] = posición en el ranking del i-ésimo match
    precision_result = np.argwhere(mask == True)
    precision_result = precision_result.reshape(
        precision_result.shape[0])

    # Si hay al menos un positivo:
    if precision_result.shape[0] != 0:

        # Marcar con 1 desde el primer match en adelante
        # cmc[k] = 1 significa "el match aparece en posición <= k"
        cmc[int(precision_result[0]):] = 1

        # d_recall: cuánto aumenta el recall por cada positivo
        # 1/N donde N = total de positivos en la gallery
        d_recall = 1.0 / len(precision_result)

        # Acumular AP con regla del trapecio
        ap = 0
        for i in range(len(precision_result)):

            # Precisión en la posición del i-ésimo match
            # = (matches hasta aquí) / (rank del match)
            precision = (i + 1) * 1.0 / (precision_result[i] + 1)

            # Precisión del match anterior (para el trapecio)
            if precision_result[i] != 0:
                old_precision = i * 1.0 / precision_result[i]
            else:
                # Caso especial: primer match en posición 0
                old_precision = 1.0

            # Área del trapecio entre (old_prec, precision) con altura d_recall
```

```

    ap += d_recall * (old_precision + precision) / 2

    return ap, cmc

else:
    # No hay positivos para esta query (no debería pasar)
    return None, None

```

Detalles a tener en cuenta

- Si la imagen correcta aparece en la posición 1, $AP = 1.0$ (perfect ranking).
- Si aparece en la última posición, AP es bajo pero no 0 (porque se premia que aparezca *al final*, no totalmente ausente).
- Si una query tiene *un solo* positivo en la gallery, AP reduce a Top-1 (porque el recall siempre crece igual).
- Si tiene varios positivos, AP captura mejor la calidad que Top-1.

13.4 mAP (mean Average Precision)

Qué es

mAP es simplemente el promedio de AP sobre todos los queries:

$$\text{mAP} = \frac{1}{N_q} \sum_{i=1}^{N_q} \text{AP}_i \quad (13.4)$$

donde N_q es el número de queries (textos).

Por qué mAP y no sólo Top-1

- **Top-1** sólo cuenta si el primero es correcto. Ignora si los otros positivos están bien rankeados.
- **mAP** considera el ranking *completo*: si una query tiene 5 positivos y todos están en los primeros 5, $AP = 1.0$. Si dos están fuera del top-5, $AP < 1.0$.

En este proyecto cada query (texto) tiene *un solo* positivo en la gallery (la imagen correspondiente). En ese caso mAP y Top-1 están correlacionados pero NO son iguales:

- Top-1 cuenta cuántos queries tienen el positivo en posición 1.
- mAP promedia la calidad del ranking: un query con el positivo en posición 2 da $AP = 0.5$ (en el caso ideal), mientras que Top-1 = 0 para ese query.

13.5 CMC: el vector completo

Qué es

El proyecto devuelve **cmc** que es un vector de tamaño N_{gallery} . Para cada posición k :

$$\text{cmc}[k] = \frac{1}{N_q} \sum_{i=1}^{N_q} \mathbb{I}[\text{rank}_i \leq k] \quad (13.5)$$

Es la fracción acumulada de queries cuyo positivo aparece en posición $\leq k$. $\text{cmc}[0] = \text{Top-1}$, $\text{cmc}[4] = \text{Top-5}$, etc.

Implementación

```
def evaluate(similarity, label_query, label_gallery):
    """
    similarity:      matriz (N_img, N_txt) de similitud coseno
    label_query:     vector (N_txt,) con pids de los textos (queries)
    label_gallery:   vector (N_img,) con pids de las imágenes (gallery)
    Returns: (cmc, mAP)
    """

    # Convertir de torch.Tensor a numpy array
    label_query = label_query.numpy()
    label_gallery = label_gallery.numpy()

    # Vector acumulador de CMC
    # Tamaño = número de imágenes en la gallery
    # Cada posición k almacena cuántos queries acertaron en top-k
    cmc = np.zeros(label_gallery.shape)

    # Acumulador de AP (para luego sacar promedio = mAP)
    ap = 0

    # Para cada query de texto:
    for i in range(len(label_query)):

        # Calcular AP para esta query
        # similarity[:, i] = columna i = similitud de TODAS las imágenes
        #                  con el texto i-ésimo (orden: filas son imágenes)
        ap_i, cmc_i = calculate_ap(
            similarity[:, i],
            label_query[i],
            label_gallery)

        # Acumular cmc (suma de indicadores)
        cmc += cmc_i

        # Acumular AP
        ap += ap_i

    # Normalizar: cmc[k] = fracción de queries que acertaron en top-k
    cmc = cmc / len(label_query)

    # Normalizar: mAP = promedio de AP sobre todos los queries
    map = ap / len(label_query)

    """
    Comentario sobre cmc:
    cmc[i] es un vector [0, 0, ..., 1, 1, ..., 1] donde el primer 1
    está en la posición del primer match correcto. rank-n y todos
    los ranks posteriores suman un acierto. Al sumar todos los
    vectores y dividir por n_query obtenemos el rank-k accuracy.
    """

    return cmc, map
```

13.6 Interpretación de números

Resultados típicos para CUHK-PEDES

Tabla 13.2: Resultados esperados según el README y el paper.

Setup	Top-1	Top-5	Top-10	mAP
Sin entrenar (modelo random)	~0.02 %	0.05 %	0.10 %	0.10 %
HF (128×128) 50 batches	0.02 %	0.08 %	0.17 %	0.10 %
HF entrenado completo	45–52 %	~70 %	~80 %	~40 %
Original paper (paper)	54.02 %	75.51 %	85.28 %	~50 %

Cómo interpretar un resultado “Top-1 = 50 %”

Dice que, dado un texto al azar del test set, en el 50 % de los casos la imagen correcta aparece en la primera posición del ranking.

- Es “bueno” para ReID por texto (un humano hacerlo mejor).
- Es “malo” comparado con face recognition (>99 %).
- El gap entre Top-1 y Top-5 indica cuántas “casi correctas” hay en el top-5: si Top-5 = 75 % y Top-1 = 50 %, el 25 % restante tiene el correcto en posiciones 2–5.

Cómo interpretar mAP

- mAP es siempre **menor o igual** a Top-1 (matemáticamente: $mAP \leq cmc[0]$).
- Si Top-1 $\approx$ mAP, el modelo es “perfecto” en sus predicciones (los positivos casi siempre están en posición 1).
- Si Top-1 > mAP · 2, hay muchas predicciones donde el correcto está en posiciones 2–5 (modelo “casi bueno”).

13.7 Ejemplo numérico completo

Supongamos 3 queries y 5 imágenes en gallery. La matriz de similitud es:

	Q1	Q2	Q3
img1	[0.9	0.1	0.3]
img2	[0.5	0.8	0.2]
img3	[0.2	0.6	0.7]
img4	[0.7	0.3	0.5]
img5	[0.4	0.5	0.4]

y los pids (queries vs gallery):

- Q1 busca img2 (pid=2)
- Q2 busca img3 (pid=3)
- Q3 busca img3 (pid=3)

Calcular Top-1

Para cada query ordenamos la columna de similitud:

- **Q1:** img1 (0.9), img4 (0.7), img2 (0.5), img5 (0.4), img3 (0.2). Correcto (img2) en posición **3**. Top-1: NO.
- **Q2:** img2 (0.8), img3 (0.6), img5 (0.5), img4 (0.3), img1 (0.1). Correcto (img3) en posición **2**. Top-1: NO.

- **Q3:** img3 (0.7), img4 (0.5), img1 (0.3), img2 (0.2), img5 (0.4) Correcto (img3) en posición **1**.
Top-1: **SÍ**.
Top-1 = 1/3 = 33.3 %.

Calcular mAP

- **AP(Q1):** positivo en posición 3 de 5. $AP = 1/3 \cdot (1 + (1/3+0)/2) = 1/3 + 1/9 = 4/9 \approx 0.44$.
 - **AP(Q2):** positivo en posición 2 de 5. $AP = 1/1 \cdot (1 + 0)/2 = 0.5$.
 - **AP(Q3):** positivo en posición 1 de 5. $AP = 1$.
- mAP = (0.44 + 0.5 + 1) / 3 $\approx$ 0.65.**

Calcular Top-5

Todos los positivos están en posiciones ≤ 5 (trivialmente), así que **Top-5 = 100 %**.

13.8 Resumen

Tabla 13.3: Resumen de las cuatro métricas.

Métrica	Qué captura
Top-1 (cmc[0])	¿La imagen correcta es la #1?
Top-5 (cmc[4])	¿Está en el top 5?
Top-10 (cmc[9])	¿Está en el top 10?
mAP	Calidad global del ranking

Consejo para reportar resultados: siempre reportar al menos Top-1, Top-5, Top-10 y mAP. Top-1 solo es insuficiente porque no captura la calidad del ranking más allá del primer lugar.

Capítulo 14

Apéndices

14.1 Glosario

BERT	<i>Bidirectional Encoder Representations from Transformers</i> . Modelo de lenguaje preentrenado de Google (2018). En este proyecto se usa sólo como tokenizer.
BiGRU	GRU bidireccional. Variante de RNN que procesa la secuencia en ambas direcciones y concatena los estados ocultos.
CBIR	<i>Content-Based Image Retrieval</i> . Recuperación de imágenes por contenido visual en lugar de metadatos.
CMC	<i>Cumulative Matching Characteristic</i> . Curva que mide la fracción de queries cuyo positivo aparece en el top-k. $\text{cmc}[\mathbf{k}] = \text{Top}-(\mathbf{k}+1)$ accuracy.
CUHK-PEDES	<i>CUHK Person Description Dataset</i> . Benchmark estándar de re-ID por texto, 40 206 imágenes con descripciones.
DSC	<i>Depthwise Separable Convolution</i> . Factorización de una convolución estándar en depthwise (por canal) + pointwise (1×1). Reduce parámetros y FLOPs.
EfficientNet-B0	Familia de CNNs con compound scaling. B0 es la variante más pequeña (~5,3M parámetros), preentrenada en ImageNet.
FPN	<i>Feature Pyramid Network</i> . Estructura que combina features de múltiples escalas para detección.
GRU	<i>Gated Recurrent Unit</i> . RNN con dos compuertas (reset, update), más simple que LSTM.
mAP	<i>mean Average Precision</i> . Promedio de AP sobre todas las queries. Mide calidad del ranking completo.
NMS	<i>Non-Maximum Suppression</i> . Técnica para eliminar detecciones redundantes.
Person Re-ID	Tarea de reconocer a la misma persona en diferentes cámaras o momentos.
Triplet loss	Pérdida que enforce que un anchor esté más cerca de un positivo que de un negativo, con un margen dado.
U-TextReIDNet	<i>Unified Text ReID Network</i> . Modelo del paper que une detección humana con re-ID por texto.
Top-k	Fracción de queries cuyo positivo aparece entre los primeros k resultados.

14.2 Tabla maestra de parámetros del modelo

Capa	Forma de salida	Parámetros
Embedding(29610, 512)	((B, 100, 512))	15,16 M
Dropout(0.30)	((B, 100, 512))	0
BiGRU(512, 1024, bidir)	((B, 100, 2048))	4,72 M
Suma fwd+bwd / 2	((B, 100, 1024))	0
Permute + unsqueeze	((B, 1024, 100, 1))	0
EfficientNet-B0	((B, 1280, 16, 16))	5,29 M
DSC(1280, 1024)	((B, 1024, 16, 16))	~14 K
AdaptiveMaxPool2d(1, 1)	((B, 1024, 1, 1))	0
DSC(1024, 1024)	((B, 1024, 1, 1))	~10 K
Classifier(1024, num_classes)	((B, num_classes))	~11 K por clase
Total (sin clasificador)	—	~25,2 M
Total (con clasificador)	—	~25,2 M + 1024 · N_c

Tabla 14.1: Recuento completo de parámetros. N_c = número de identidades (clases) en el dataset de entrenamiento. Para CUHK-PEDES, $N_c = 11\,003$.

14.3 Tabla maestra de hiperparámetros de entrenamiento

Hiperparámetro	Valor	Ubicación
Épocas	60	config.py: 71
Batch size (default)	8	config.py: 123,129
Batch size (Jetson)	4	config.py: 126
Learning rate inicial	1e-3	config.py: 75
Weight decay	(decoupled, AdamW)	train.py: 98
β_1 AdamW	0.90	config.py: 72
β_2 AdamW	0.999	config.py: 73
LR decay epochs	[20, 40]	config.py: 74
LR decay factor	0.1 (default PyTorch)	(implícito)
Mixed precision (GPU)	FP16	train.py: 130
Mixed precision (CPU)	BF16	train.py: 130
Ranking loss weight α	1.0	config.py: 87
Identity loss weight β	1.0	config.py: 88
Triplet margin	0.5	config.py: 86
Seed	3407	config.py: 79

Tabla 14.2: Hiperparámetros de entrenamiento.

14.4 Resultados reportados en el paper

Dataset	Top-1	Top-5	Top-10
CUHK-PEDES	54,02 %	75,51 %	85,28 %
RSTPReid	38,45 %	—	—

Tabla 14.3: Resultados del paper original.

U-TextReIDNet: 38,77 M parámetros totales.

14.5 Mapa de archivos

```

textreid-train/
|-- README.md
|-- config.py           # Configuración global
|-- train.py           # Script de entrenamiento
|-- evaluate.py        # Script de evaluación
|-- requirements.txt
|
|-- model/
|   |-- textreidnet.py  # Modelo principal
|   |-- visual_network.py # EfficientNet-B0 wrapper
|   |-- language_network.py # BERT tokenizer + BiGRU
|   |-- efficientnet_backbone.py # Variante multi-stage (no usada)
|   |-- model_utils.py  # DepthwiseSeparableConv, etc.
|
|-- datasets/
|   |-- bases.py        # ImageTextDataset, etc.
|   |-- cuhkpedes.py    # Loader dataset original
|   |-- cuhkpedes_hf.py # Loader dataset HuggingFace
|   |-- cuhkpedes_dataloader.py # Constructor de DataLoaders
|   |-- bert_tokenizer.py # BERT tokenizer
|   |-- simple_tokenizer.py # CLIP-style BPE tokenizer
|   |-- tiktoken_tokenizer.py # OpenAI tiktoken
|
|-- evaluation/
|   |-- ranking_loss.py # Triplet semi-hard
|   |-- identity_loss.py # Cross-entropy simétrica
|   |-- focal_loss.py   # Sigmoid focal (no usada)
|   |-- evaluations.py  # Top-k, mAP
|
|-- utils/
|   |-- iotools.py      # read_image, read_json
|   |-- miscellaneous_utils.py # set_seed, pad_tokens, etc.
|
|-- scripts/
|   |-- download_hf_dataset.py # Descarga y convierte HF
|
|-- data/
|   |-- CUHK-PEDES-HF/    # Dataset descargado

```

```

|         |-- raw.parquet
|         |-- reid_raw.json
|         `-- imgs/all/*.jpg
|
|-- docs/
|   `-- paper.md                # Paper traducido al español
|
|-- notebooks/
|   `-- 01_lab_validate_dataset.ipynb
|
`-- book/                        # (este libro)
    |-- main.tex
    |-- chapters/*.tex
    `-- figures/*.tikz

```

14.6 Cheatsheet de Bash

```

1  # Instalar dependencias
2  pip install -r requirements.txt --extra-index-url https://download.pytorch.org/whl/cu117
3
4  # Descargar dataset
5  python scripts/download_hf_dataset.py
6
7  # Entrenar
8  python train.py --dataset_source huggingface --epochs 60
9
10 # Evaluar
11 python evaluate.py \
12     --checkpoint data/checkpoints/TextReIDNet_latest.pth.tar \
13     --dataset_source huggingface \
14     --split test
15
16 # Inspeccionar configuración
17 python -c "from config import sys_configuration; \
18     cfg = sys_configuration(); print(cfg)"

```

14.7 Referencias

- Agyeman, R. & Rinner, B. (2024). *Decentralized Text-Based Person Re-Identification in Multicamera Networks*. IEEE Access. DOI: 10.1109/ACCESS.2024.3501382. Traducción al español en docs/paper.md.
- Tan, M. & Le, Q. V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. ICML. arXiv:1905.11946.
- Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL. arXiv:1810.04805.
- Cho, K. et al. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. EMNLP.
- Howard, A. G. et al. (2017). *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. arXiv:1704.04861.
- Li, S., Xiao, T., Li, H., Zhou, B., Yue, D. & Wang, X. (2017). *Person Search with Natural Language Description*. ICCV.
- Loshchilov, I. & Hutter, F. (2019). *Decoupled Weight Decay Regularization*. ICLR. (AdamW).

- **Lin, T.-Y., Goyal, P., Girshick, R., He, K. & Dollár, P.** (2017). *Focal Loss for Dense Object Detection*. ICCV. (RetinaNet).